Syrian Private University
Faculty of Artificial Intelligence
Engineering

Enhanced Identity-Based Delegated Data Integrity Auditing for Secure Cloud Storage Using Digital Signatures and SIEM

Supervised by :
Wassim Aljuneidi

Prepared by :
Ruba Tadmory & Nawar Alani

Table of contents

Introduction **8**
1.1 Introduction 9
1.2 Important Applications 10
1.3 Challenges 12
1.4 Objectives 14
1.5 Previous Studies 16
1.6 Research Gap 17
Chapter One: Theoretical Study **19**
2.1 Cloud Computing 20
2.2 Cloud Storage 20
2.3 Data Integrity 21
2.4 Data Integrity Auditing 22
2.5 Identity-Based Controlled Delegation 22
2.6 Digital Signatures 23
2.7 Trusted Evidence and Non-Repudiation 24
2.8 Security Information and Event Management (SIEM) 24
2.9 Wazuh 25
2.10 Nextcloud 26
Chapter Two **27**
3.1 Selected Work Environment 28
3.2 Installation of the Working Environment 30
3.2.1 Installing VMware Workstation 31
3.2.2 Installing Ubuntu Server 32
3.2.3 Installing Wazuh SIEM 34
3.2.4 Configuring the Environment 36
3.2.5 Verifying the Installation 38
3.3 Proposed Framework Operation 39
3.3.1 Phase One: Data Preparation 39
3.3.2 Hash Generation 40
3.3.3 Digital Signature Generation 40
3.3.4 Secure File Upload 41
3.3.5 Evidence Exchange 42
3.3.6 Continuous Security Monitoring using SIEM 42

3.3.7 File Retrieval and Integrity Verification 43
3.3.8 Experimental Environment 44
3.3.9 Functional Validation of the Proposed Environment 45
3.3.10 User Authentication 45
3.4 Proposed Controlled Delegation Authentication Protocol (CDAP) 52
3.4.1 Entities and Assumptions 54
3.4.2 Phase One: Creating the Delegation 54
3.4.3 Phase Two: Uploading a File Using the Delegation 57
3.4.4 Revoking a Delegation 61
3.4.5 Integration with Wazuh SIEM 61
3.4.6 Security Analysis of CDAP 62
Chapter Three: Practical Implementation 63
4.1 Experimental Environment 64
4.2 Cloud Storage Deployment 64
4.3 Digital Signature Implementation 66
4.4 Trusted Third Party (TTP) and Resolve Mode 67
4.5 Broadcast Session Implementation 69
4.6 Non-Repudiation Receipt (NRR) Implementation 70
4.7 Download Verification 72
4.8 Rollback Attack Simulation 74
4.9 Wazuh SIEM Monitoring and Alert Generation 75
4.10 Overall Experimental Scenario 76
Chapter Four: Results and Discussion 78
5.1 Experimental Results 79
5.2 Security Evaluation Before and After Applying the Proposed Framework 79
5.3 Discussion 80
5.4 Overall Achievement of the Proposed Framework 81
Conclusion 82
References 84

List of figures

Figure 1: Cloud Computing Architecture 12
Figure 2: Proposed Work Environment 29
Figure 3: VMware Workstation Interface 32
Figure 4: Ubuntu Server Installation in VMware Workstation 34
Figure 5: Wazuh SIEM Architecture 36
Figure 6: Overall Workflow of the Proposed Cloud Storage Security Framework 51
Figure 7: CDAP Workflow Showing Both Phases and the Rejection Paths 53
Figure 8: Phase One – Creating and Registering a Delegation 56
Figure 9: Standard Upload by a Regular User without Delegation 57
Figure 10: Phase Two – Delegated Upload by User B 60
Figure 11: Comparison Between Traditional Cloud Storage and the Proposed MPNR Framework 81

List of tables

Table 1.1: Information Security Goals..... 15
Table 1.2: Previous Studies..... 17
Table 2.1: Components of the Proposed Work Environment..... 30
Table 2.2: Software Components Used in the Project..... 30
Table 2.3: Minimum Hardware Requirements for Ubuntu Server..... 33
Table 3.1: Entities Involved in CDAP and Their Roles……45
Table 3.2: Security Events Generated by the CDAP Delegation Service……..61
Table 3.3: Security Threats and Countermeasures in CDAP…….62
Table 5.1: Security Comparison Between Traditional Cloud Storage and the Proposed MPNR Framework..... 80

List of Abbreviation

Abbreviation	Full Form
NR	Non-Repudiation
TPNR	Two-Party Non-Repudiation
MPNR	Multi-Party Non-Repudiation
NRO	Non-Repudiation of Origin
NRR	Non-Repudiation of Receipt
NROOC	Non-Repudiation of Origin for Owner-Cloud communication
NROOU	Non-Repudiation of Origin for Owner-User communication
NRRCO	Non-Repudiation of Receipt from Cloud provider to Owner
NRRUiO	Non-Repudiation of Receipt from User to Owner
NRRCUi	Non-Repudiation of Receipt from Cloud provider to User
TTP	Trusted Third Party
SWMR	Single Writer Multiple Reader
MWMR	Multiple Writer Multiple Reader
ACL	Access Control List
MIMT	Man-In-The-Middle
MD5	Message Digest Algorithm 5
SHA	Secure Hash Algorithm
AES	Advanced Encryption Standard
PBES	Proxy-Based Encryption Scheme
DRM	Digital Rights Management
CSMA/CD	Carrier Sense Multiple Access with Collision Detection
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
GB	Gigabyte
TB	Terabyte

List of symbols

Symbol	Meaning
O	Owner (Data Owner)
C	Cloud Provider
U	User
Ui	User number i
H	Hash Function
H(Data)	Hash value of the data
K	Encryption Key
PK	Public Key
SK	Private / Secret Key
S	Digital Signature
E(K)	Encryption using key k
D(K)	Decryption using key k
L	Label / Transaction Identifier
Seq	Sequence Number
T	Time Limit / Timeout
Tg	Time Tag
Tx	Expiration Time
Chain_G	Hash Chain used for linking data versions
G	Group encryption parameter
EGU	Encrypted group information for users
EO	Evidence related to Owner
EC	Evidence related to Cloud Provider
A	Arbitrator / Entity involved in resolving disputes
B	Recipient/User group in communication
M	Message / Data content
Hash(Data)	Digital fingerprint of data
Version	Data version number

Introduction

1.1 Introduction

In recent years, cloud computing has emerged as one of the most influential technologies in the field of information technology, fundamentally transforming the way computing resources, applications, and data are managed and delivered. By providing on-demand access to computing resources and services over the Internet, cloud computing enables organizations and individuals to reduce infrastructure costs, improve operational efficiency, and achieve greater scalability and flexibility. Among its various applications, cloud storage has become one of the most widely adopted cloud services due to its ability to store, manage, access, and share large volumes of data efficiently *[2]*.

The rapid growth of digital transformation, together with the increasing adoption of Big Data, Artificial Intelligence (AI), the Internet of Things (IoT), and mobile computing, has significantly increased the demand for secure and reliable cloud storage solutions. Cloud storage is widely utilized in healthcare, education, banking, government institutions, research organizations, and business enterprises, where the security and reliability of stored data are essential requirements. Consequently, protecting cloud-stored information against unauthorized access, modification, and other security threats has become a major concern for both cloud service providers and data owners *[2]*.

Although cloud storage provides significant benefits, outsourcing data to third-party cloud service providers introduces several security challenges. Since data owners no longer maintain direct control over their data and may rely on other users or authorized entities to perform operations on their behalf, ensuring data integrity and controlling delegated access become increasingly important. In particular, when a data owner delegates data-related operations to another entity, the cloud environment must be able to verify the identity of the delegated entity, enforce the authorized scope of delegation, and ensure that delegated operations do not compromise the integrity of the stored data *[1][2][3]* .

To address these challenges, researchers have proposed various cryptographic and auditing mechanisms for verifying the integrity of outsourced cloud data. These approaches commonly employ cryptographic techniques such as hash functions, digital signatures, identity-based mechanisms, and third-party auditing to enable efficient verification without requiring the data owner to continuously perform the auditing process. Identity-based controlled delegation provides an additional security mechanism by allowing a data owner to authorize another entity to perform specific operations while maintaining control over the delegated privileges. This is particularly important in cloud environments where data management tasks may need to be performed by authorized proxies or service entities on behalf of the original data owner *[1][2][3].*

This project is based on the research paper entitled *Identity-based controlled delegated outsourcing data integrity auditing scheme*, which proposes an identity-based auditing

framework for cloud data that supports controlled delegation and outsourced integrity auditing. The proposed scheme addresses important security requirements related to delegation control, auditing correctness, and privacy preservation in cloud storage environments. By adopting the core concept of controlled delegated data integrity auditing, this project extends the cloud storage security framework previously developed in the Applications Project *[3]*.

The proposed development integrates the identity-based controlled delegation and data integrity auditing concepts with the existing security mechanisms of the previous project, including digital signatures, trusted evidence management, and Security Information and Event Management (SIEM)*[3]*. Digital signatures are used to provide cryptographic evidence of data authenticity and integrity*[1][7]*, while the trusted evidence mechanism supports the preservation of security-related information for verification and dispute analysis*[1]*.

Furthermore, the integration of the Wazuh SIEM platform enables centralized security monitoring, event collection, alert generation, and detection of suspicious activities related to delegated operations and data auditing *[9]*.

Therefore, the proposed framework aims to provide a more comprehensive cloud storage security environment by combining controlled identity-based delegation, data integrity auditing, digital signatures, trusted evidence, and SIEM-based security monitoring. This development extends the capabilities of the previous Applications Project while addressing the additional security challenges introduced when cloud data management and auditing operations are delegated to authorized entities.

1.2 Important Applications

Cloud storage has become one of the most important applications of cloud computing due to its ability to provide scalable, flexible, and cost-effective storage services. Individuals and organizations rely on cloud storage to manage large volumes of data while benefiting from high availability, remote accessibility, data sharing, and efficient data management. Unlike traditional storage systems, cloud storage enables users to access their data from different locations through Internet-connected devices, making it an essential technology in today's digital environment *[1][2]*.

Cloud storage services are widely adopted across various sectors, including healthcare, education, finance, government, manufacturing, and enterprise environments. In the healthcare sector, cloud storage facilitates the storage and sharing of electronic medical records among authorized healthcare providers. In education, it supports online learning platforms by enabling students and instructors to access educational resources remotely. Financial institutions use cloud storage to manage customer information, transaction records, and backup systems, while government organizations utilize cloud services to improve data accessibility, collaboration, and operational efficiency.

The increasing growth of Big Data, Internet of Things (IoT) devices, Artificial Intelligence (AI), and digital transformation initiatives has further increased the demand for secure and reliable cloud storage solutions*[2].* As organizations continue to migrate critical information and business applications to cloud environments, ensuring data integrity, confidentiality, availability, authentication, authorization, and accountability has become a fundamental requirement for maintaining trust and protecting valuable digital assets*[2][8]*.

In addition to supporting data storage and collaboration, cloud environments increasingly require authorized users or external entities to perform data-related operations on behalf of data owners. This delegation can improve operational efficiency by allowing trusted users or service entities to manage, upload, update, or audit data without requiring the original data owner to perform every operation personally. However, delegated operations introduce additional security requirements, particularly regarding the identity of the delegated entity, the scope of its authorized privileges, and the integrity of the data involved in these operations*[3].*

Therefore, secure cloud storage solutions must not only provide efficient data access and management but also support reliable mechanisms for controlling delegated operations and verifying the integrity of outsourced data. Integrating identity-based delegation with data integrity auditing can help ensure that only authorized entities perform permitted operations while enabling the cloud-stored data to be independently verified. These capabilities are particularly important in modern cloud environments where data management responsibilities are increasingly distributed among multiple users and service entities *[2][3]* .

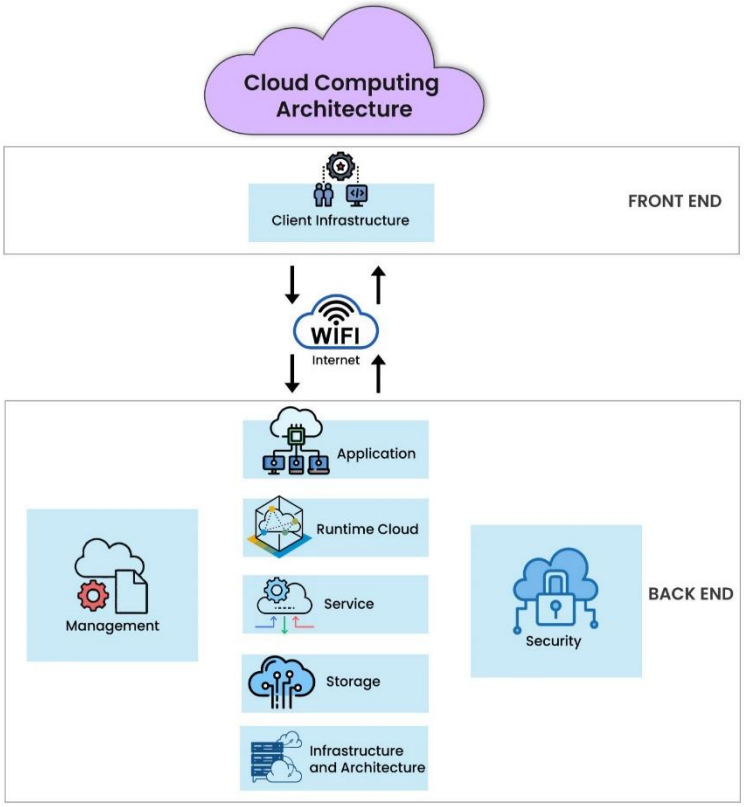


Figure 1: cloud computing architecture

1.3 Challenges

Despite the numerous advantages offered by cloud storage, it introduces several security and privacy challenges due to the outsourcing of data to third-party Cloud Service Providers (CSPs). Since data owners no longer maintain direct physical control over their stored information, ensuring data integrity, privacy, authentication, authorization, and availability becomes a major concern. The centralized storage of large amounts of user data may also make cloud environments attractive targets for attackers, increasing the potential impact of unauthorized access, data leakage, and data tampering *[1][2]*.

One of the most significant challenges in cloud storage is maintaining **data integrity**. Data stored in the cloud should remain complete, accurate, and unaltered throughout its lifecycle. Unauthorized modification, deletion, or corruption of outsourced data can affect the reliability of cloud services and may result in financial, operational, or legal consequences. Therefore, cloud storage systems require efficient integrity auditing mechanisms that enable the correctness of outsourced data to be verified without requiring the data owner to continuously maintain direct control over the stored information *[2][3]*.

Another important challenge arises when data management or auditing operations are

outsourced to another entity. In practical cloud environments, a data owner may need to appoint a proxy or authorized user to perform operations such as uploading, modifying, or managing data on their behalf. However, uncontrolled delegation may introduce additional security risks. The system must be able to determine **who is authorized to act on behalf of the data owner, what operations that entity is allowed to perform, and whether the delegated operations remain within the permitted scope**. The selected research paper identifies the lack of controlled outsourced delegation as an important limitation in existing cloud data auditing solutions and proposes an identity-based controlled delegation mechanism to address this issue *[3]*.

Identity management and authorization therefore represent another major challenge in delegated cloud environments. Traditional certificate-based approaches may introduce additional management overhead, particularly in distributed environments involving multiple users and entities. An identity-based approach can simplify the authentication and authorization of delegated entities by using recognizable identities to determine whether a proxy is authorized to perform a specific operation. However, the delegation mechanism must still ensure that unauthorized entities cannot perform operations on behalf of the data owner *[3]*.

In addition, cloud data may undergo **dynamic operations**, including data updates and structural changes. Maintaining data integrity during these operations can become computationally challenging, particularly when large amounts of data are involved. Efficient data structures and auditing mechanisms are therefore required to support dynamic updates while maintaining reliable integrity verification. The selected scheme addresses this challenge through a bucket-based Red-Black Tree structure designed to support efficient dynamic data operations and auditing *[2][3]*.

Furthermore, cryptographic integrity and authorization mechanisms alone do not provide continuous visibility into the security events occurring within the cloud environment. Even when a delegated operation is rejected or an integrity verification fails, security administrators may need centralized records and alerts to identify suspicious behavior, repeated unauthorized attempts, or abnormal activities. Integrating a **Security Information and Event Management (SIEM)** system can complement these cryptographic mechanisms by collecting security events, monitoring delegated operations, correlating relevant activities, and generating alerts for suspicious behavior *[9]*.

Therefore, the main challenge addressed by this project is not only how to verify the integrity of outsourced cloud data, but also how to **control delegated operations according to user identity and authorization while continuously monitoring the resulting security activities**. Combining identity-based controlled delegation and data integrity auditing with the digital signatures, trusted evidence mechanisms, and SIEM capabilities inherited from the previous Applications Project provides a more comprehensive approach to securing delegated cloud storage operations.

1.4 objectives

The primary objective of this project is to investigate the security challenges associated with delegated cloud storage and outsourced data integrity auditing. The project aims to develop an enhanced cloud storage security framework based on identity-based controlled delegation and data integrity auditing, while extending the security mechanisms implemented in the previous Applications Project through the integration of digital signatures, trusted evidence management, and Security Information and Event Management (SIEM).

The specific objectives of this project are summarized as follows:

1. To study the concept of cloud storage and identify its main security requirements.
2. To analyze the importance of data integrity auditing in outsourced cloud storage environments.
3. To study the identity-based controlled delegation mechanism proposed for outsourced **cloud** data management and auditing.
4. To examine how identity-based authentication and authorization can be used to control delegated operations performed by authorized proxies.
5. To implement a mechanism for verifying the integrity of cloud-stored data during delegated and outsourced operations.
6. To integrate digital signatures and trusted evidence mechanisms from the previous Applications Project to provide cryptographic evidence of data authenticity and integrity.
7. To integrate a Security Information and Event Management (SIEM) system using Wazuh for monitoring delegated operations, integrity auditing events, and suspicious security activities.
8. To evaluate how the integration of controlled delegation, data integrity auditing, digital signatures, trusted evidence, and SIEM enhances the security and monitoring capabilities of the cloud storage environment.

Security Goal	***Purpose***	***Common Technologies***
Confidentiality	*Prevent unauthorized disclosure of information*	*Access Control*
Integrity	*Ensure that information remains accurate, complete, and unaltered*	*Hash Functions, Digital Signatures*
Availability	*Ensure that information and services are accessible when required*	*Backup, Redundancy, Disaster Recovery*

Authentication	*Verify the identity of users or entities*	*Passwords, Digital Certificates, Identity-Based Cryptography*
Authorization	*Control which operations an authenticated entity is permitted to perform*	*Access Control, Role-Based Access Control, Delegation Policies*
Data Integrity Auditing	*Verify the correctness and integrity of outsourced cloud data*	*Integrity Verification, Auditing Protocols, Hash-Based Mechanisms*
Non-Repudiation	*Provide evidence that prevents an entity from credibly denying a performed action*	*Digital Signature, Audit Evidence*
Accountability	*Record and monitor security-related activities to support investigation and incident response*	*Audit Logs, SIEM*

Table 1.1 Information Security Goals *[7]*.

1.5 Previous Studies

Ref	*Year*	*Study Title*	*Main Contribution*	*Relation to our project*
[1]	*2024*	**Investigation on Storage-Level Data Integrity Strategies in Cloud Computing: Classification, Security Obstructions, Challenges and Vulnerability**	Provides a comprehensive review and classification of **data integrity protection and auditing strategies** in cloud storage, including security threats, vulnerabilities, and existing integrity verification mechanisms.	Provides the theoretical background for the **Data Integrity** component of our project and helps identify suitable integrity verification and auditing mechanisms.
[2]	*2024*	**Identity-Based Controlled Delegated Outsourcing Data Integrity Auditing Scheme**	Proposes an identity-based controlled delegation mechanism that allows authorized proxies to perform data outsourcing operations on behalf of the data owner, while supporting authentication, authorization, integrity auditing, and dynamic data updates.	**Forms the main research basis of our project.** Its controlled delegation and identity-based auditing concepts are integrated with the security mechanisms inherited from our previous Applications Project.
[3]	*2024*	**Sec-Auditor: A Blockchain-Based Data Auditing Solution for Ensuring Integrity and Semantic Correctness**	Proposes a blockchain-based auditing solution for verifying data integrity and semantic correctness in cloud storage while maintaining constant communication cost independent of the audited data volume.	Provides a reference for advanced cloud auditing and tamper-resistant audit records. Blockchain-based auditing is considered a **potential future extension**, rather than a core component of the current project.
[4]	*2025*	**An Improved Blockchain-Based Cloud Auditing Scheme Using Dynamic Aggregate Signatures**	Proposes an improved blockchain-based cloud auditing scheme using multiple hashing, dynamic aggregate signatures, BLS signatures, and random challenges to strengthen integrity protection and resistance against forgery, tampering, and deletion attacks.	Provides a reference for advanced combinations of **hashing, digital signatures, and cloud auditing**. Blockchain and aggregate signatures are considered potential future extensions to the current framework.
[5]	*2025*	**Integrated Protocol for Verifiable Confidential**	Proposes an integrated protocol that combines verification of	Provides a potential direction for extending the current framework

		Cloud Computing and Data Integrity Auditing	cloud-computing results, cloud data integrity auditing, and privacy protection in a confidential cloud-computing environment.	with **confidentiality and privacy-preserving mechanisms** while maintaining its existing integrity auditing and security monitoring capabilities.

Table 1.2 Previous Studies

1.6 Research Gap

The reviewed studies demonstrate that cloud data security has been extensively investigated from different perspectives, including data integrity protection, public auditing, controlled delegation, blockchain-based auditing, privacy preservation, and cryptographic verification. However, these approaches generally focus on specific security requirements rather than providing an integrated framework that combines delegated data management, cryptographic evidence, and continuous security monitoring within a practical cloud storage environment *[2][6]*.

The study presented in [1] provides a comprehensive investigation and classification of cloud data integrity strategies and identifies several security challenges and vulnerabilities. However, as a review-based study, it does not provide an integrated implementation that combines controlled delegation, cryptographic evidence, and SIEM-based monitoring in a practical cloud storage environment *[2]*.

The study in [2], which forms the primary research basis of this project, addresses the problem of controlled delegated outsourcing data integrity auditing through an identity-based mechanism. The scheme enables a data owner to authorize a specific proxy to perform selected data outsourcing operations and supports integrity auditing and dynamic data operations *[3]*. However, the proposed approach does not incorporate the SIEM-based security monitoring and trusted evidence mechanisms developed in our previous Applications Project. Therefore, there is an opportunity to extend the controlled delegation and auditing concept with additional monitoring and security-evidence capabilities .

The blockchain-based approaches presented in [3] and [4] demonstrate the benefits of tamper-resistant audit records, advanced cryptographic techniques, and decentralized verification *[4][5]*. Nevertheless, these approaches introduce blockchain-based architectures that are different from the cloud storage environment implemented in our previous project. They therefore provide useful directions for future development but do not directly address the integration of identity-based controlled delegation with the existing digital signature, trusted evidence, and SIEM mechanisms.

The protocol presented in [5] focuses on combining confidential cloud computing with data integrity auditing and privacy protection *[6]*. Although it provides an advanced approach to

protecting cloud data and verifying cloud-computing results, its main focus differs from the controlled delegation and security-event monitoring requirements considered in this project.

Based on the comparison of the reviewed studies, a clear **integration gap** can be identified. Existing approaches address individual or selected aspects of cloud security, such as integrity auditing, controlled delegation, blockchain-based auditing, privacy protection, or cryptographic verification *[2][6]*. However, there is a need for a practical framework that combines **identity-based controlled delegation and data integrity auditing** with **digital signatures, trusted security evidence, and SIEM-based monitoring** within a cloud storage environment.

Therefore, this project addresses this integration gap by extending the security framework developed in the previous Applications Project. The proposed framework adopts the identity-based controlled delegation and data integrity auditing concepts from [2] and integrates them with the existing digital signature, trusted evidence, and Wazuh SIEM mechanisms. This integration aims to provide not only cryptographic verification of outsourced cloud data and controlled delegation of operations, but also continuous monitoring and detection of suspicious delegated activities. In this way, the project develops the previous cloud storage security framework while addressing additional security requirements introduced by delegated data management.

CHAPTER ONE

Theoretical Study

Theoretical Study

2.1 Cloud Computing

Cloud computing has become increasingly important as a technology for providing computing resources and services. It provides users with scalable computing resources and data-center services, while reducing some of the challenges associated with traditional data administration and maintenance. The flexibility, efficiency, and relatively low cost of cloud services have contributed to the widespread adoption of cloud computing and cloud storage *[8]*.

Cloud computing also enables users to access their data from different geographical locations. In cloud storage environments, large amounts of user data are centrally maintained by Cloud Service Providers (CSPs), allowing users to benefit from convenient and flexible data storage services. However, outsourcing data to a cloud environment also introduces security concerns because users lose direct physical control over their data after it is uploaded to the cloud *[2][3]*.

As a result, protecting the integrity and security of outsourced cloud data has become an important research problem. Existing research has investigated different mechanisms for protecting and auditing cloud-stored data, including cryptographic techniques and data integrity auditing schemes *[2][3]*. These mechanisms aim to provide users with assurance that their outsourced data remains intact and has not been improperly modified or compromised.

In addition to data integrity, cloud environments may require users to delegate specific data-related operations to other entities. The selected research work addresses this requirement through an identity-based controlled delegation mechanism, in which a data owner can specify a dedicated proxy to assist with data outsourcing. The authorized proxy is authenticated and authorized according to its recognizable identity, providing controlled delegation without relying on complex certificate-management procedures *[3]*.

2.2 Cloud Storage

Cloud storage is one of the important applications of cloud computing that provides users with the ability to store and access data through cloud-based infrastructure. Cloud storage offers advantages such as flexibility, efficiency, large storage capacity, and access to data from different geographical locations. These characteristics have contributed to the increasing adoption of cloud storage by individuals and organizations*[2]*.

In a cloud storage environment, data is outsourced to a Cloud Service Provider (CSP), which is responsible for maintaining the storage infrastructure and storing users' data. This outsourcing provides users with the convenience of using cloud storage services without directly maintaining the underlying storage infrastructure. However, after data is uploaded to the cloud, the data owner no longer has direct physical control over the stored data. Consequently, ensuring that outsourced data remains intact and consistent with the original data becomes an important security requirement *[2][3]*.

Cloud storage also introduces security challenges related to data integrity and privacy. Since a CSP may centrally maintain large amounts of users' data, cloud storage environments can become attractive targets for attackers. The literature therefore identifies data integrity as an important security concern in cloud storage and discusses different cryptographic and auditing mechanisms for verifying the correctness of outsourced data *[2][3]*.

Another security requirement arises when data owners need another entity to perform data-related operations on their behalf. The selected research paper identifies controlled outsourcing as an important challenge and proposes an identity-based controlled delegation mechanism that allows a data owner to specify a dedicated proxy to assist with uploading data to the cloud. The authorized proxy is authenticated and authorized through its recognizable identity, providing controlled delegation without relying on cumbersome certificate management *[3]*.

Therefore, secure cloud storage requires mechanisms that can verify the integrity of outsourced data and control the entities that are authorized to perform delegated operations. These requirements form the basis for the identity-based controlled delegated data integrity auditing approach adopted as the main research basis of this project.

2.3 Data Integrity

Data integrity is an important security requirement in cloud storage because data owners outsource their data to third-party cloud service providers. After data is uploaded to the cloud, the data owner loses direct physical control over the stored information. Therefore, the data owner needs assurance that the outsourced data remains intact and consistent with the original data*[2][3]*.

Cloud data may be exposed to different threats that can affect its integrity, including unauthorized modification, deletion, corruption, or other forms of tampering. The security of outsourced data is therefore an important research problem, and several data integrity strategies have been proposed to allow the correctness of cloud-stored data to be verified *[2][3]*.

Cryptographic mechanisms can be used to support the verification of data integrity. The selected research paper discusses existing cloud data auditing approaches that use cryptographic techniques, including hash functions and digital signatures, to verify whether outsourced data has remained intact. However, the paper identifies that existing approaches may still face challenges related to outsourced data integrity, controlled outsourcing, and source-file auditing *[3]*.

The selected research paper addresses these challenges through an identity-based controlled delegated outsourcing data integrity auditing scheme. The proposed scheme allows the data owner to appoint an authorized proxy to assist with data outsourcing while providing mechanisms for verifying the integrity of outsourced documents and supporting dynamic data updates*[3]*.

In this project, data integrity is therefore considered a fundamental security objective. The integrity auditing component is combined with the digital-signature and trusted-evidence

mechanisms inherited from the previous Applications Project, while Wazuh is used as an additional security-monitoring layer. This integration extends the previous framework toward controlled delegated cloud data management and integrity auditing.

2.4 Data Integrity Auditing

Data integrity auditing is used to verify whether data outsourced to a cloud service provider remains correct and intact. Since the data owner does not maintain direct physical control over the data after outsourcing it to the cloud, an auditing mechanism can provide a means of checking the correctness of the stored data without requiring the data owner to continuously maintain the complete storage infrastructure. Data integrity auditing is therefore an important research area in cloud storage security. *[2][3]*

The selected research paper proposes an identity-based controlled delegated outsourcing data integrity auditing scheme that enables the integrity of outsourced data to be verified while supporting controlled delegation. The scheme involves several entities, including the Data Owner (DO), Key Generation Center (KGC), Proxy Server (PS), Third-Party Auditor (TPA), and Cloud Service Provider (CSP). These entities participate in the processes required for controlled data outsourcing and integrity auditing. *[3]*

The auditing process in the proposed scheme is based on a challenge-and-proof mechanism. The auditor generates a challenge for the outsourced data, and the corresponding proof is generated using the information maintained by the cloud-side entities. The generated proof is then verified to determine whether the outsourced data satisfies the required integrity conditions. The scheme also supports dynamic operations on outsourced data through a bucket-based Red-Black Tree (B-RBT) structure. *[3]*

The proposed auditing mechanism is designed to support controlled outsourcing while allowing the integrity of the outsourced data to be verified. The paper also considers the security and privacy requirements associated with delegated cloud data auditing and provides a security analysis and performance evaluation of the proposed scheme. *[3]*

In this project, data integrity auditing represents the main security function adopted from the selected research work. The auditing concept is integrated with the digital-signature, trusted-evidence, and SIEM mechanisms inherited from the previous Applications Project to provide a broader security framework for delegated cloud storage operations.

2.5 Identity-Based Controlled Delegation

Delegated data outsourcing allows a data owner to authorize another entity to perform selected data-related operations on the owner's behalf. Although delegation can reduce the workload of the data owner, it introduces additional security requirements because the cloud environment must determine whether the delegated entity is authorized to perform the requested operation.

Uncontrolled delegation may therefore create security risks when unauthorized entities attempt to perform operations on behalf of the data owner. *[3]*

The selected research paper introduces an identity-based controlled delegation mechanism for outsourced cloud data. The proposed scheme allows the data owner to specify a dedicated proxy to assist with data outsourcing. The authorized proxy is identified through a recognizable identity, allowing the system to authenticate and authorize the delegated entity according to the delegation requirements. *[3]*

The proposed identity-based approach is intended to reduce the management complexity associated with traditional certificate-based delegation. Instead of depending on conventional certificate management for identifying the authorized proxy, the scheme uses identity-based cryptographic mechanisms to establish controlled delegation. This allows the data owner to specify which proxy is permitted to perform the delegated outsourcing operation. *[3]*

The scheme also distinguishes the responsibilities of the different entities participating in the system. The Data Owner specifies the delegated entity, while the Proxy Server assists with the outsourcing operation. The Key Generation Center is responsible for the required key-generation functions, and the Cloud Service Provider stores the outsourced data. The Third-Party Auditor participates in the auditing process. *[3]*

In the proposed project, identity-based controlled delegation is adopted as the main development of the previous Applications Project. The controlled delegation concept is combined with the existing digital-signature and trusted-evidence mechanisms, while Wazuh SIEM is incorporated to provide security monitoring for activities associated with delegated operations and integrity auditing.

2.6 Digital Signatures

Digital signatures are cryptographic mechanisms used to provide evidence of the origin and integrity of digital data. They are based on asymmetric cryptography, where a private key is used to generate a signature and the corresponding public key is used for verification. The verification process allows a recipient to determine whether the signature is valid and whether the signed data has been altered after the signature was generated. *[7]*

Digital signatures are particularly relevant to cloud data security because outsourced data may need to be verified by entities other than the original data owner. By associating a digital signature with data, the verification process can provide cryptographic evidence related to the authenticity and integrity of the signed information. *[7]*

In the previous Applications Project, digital signatures were used as part of the non-repudiation and data-integrity mechanisms. The current project retains this mechanism as an existing security component while introducing identity-based controlled delegation and data integrity auditing as the main development.

The integration of digital signatures with the proposed framework provides an additional cryptographic layer alongside the identity-based delegation and auditing mechanisms. The integrity auditing mechanism is responsible for verifying outsourced cloud data, while digital signatures provide cryptographic evidence associated with signed data and operations.

Therefore, digital signatures in the proposed framework are not presented as a replacement for the data integrity auditing mechanism. Instead, they complement the auditing process by providing cryptographic evidence that can be used together with the trusted evidence and security-monitoring mechanisms inherited from the previous project.

2.7 Trusted Evidence and Non-Repudiation

Non-repudiation refers to the provision of evidence that can be used to associate a particular digital action or transaction with the entity responsible for that action. In cryptographic systems, digital signatures can provide evidence that a message or data item was signed using a particular private key. Such evidence can be useful when investigating disputes related to digital transactions.*[1][7]*

Trusted evidence mechanisms can be used to preserve security-related information generated during digital operations. In a cloud storage environment, such evidence may include information required to verify the authenticity and integrity of a transaction and to support the investigation of disputes.*[1]*

The previous Applications Project implemented trusted evidence mechanisms as part of its non-repudiation framework. The current project retains these mechanisms as inherited components and integrates them with the identity-based controlled delegation and data integrity auditing concepts adopted from the selected research paper.

This integration is particularly relevant to delegated operations because the system may need to distinguish between actions performed by the data owner and actions performed by an authorized delegated entity. Preserving appropriate evidence of delegated operations can therefore support subsequent verification and investigation of security-related events.

In the proposed framework, trusted evidence complements cryptographic integrity auditing and SIEM monitoring. The auditing mechanism verifies the integrity of outsourced data, while trusted evidence preserves relevant information for verification and dispute analysis. Wazuh provides an additional monitoring layer by collecting and generating security alerts for relevant system activities.

2.8 Security Information and Event Management (SIEM)

Security Information and Event Management (SIEM) is a security technology used to collect, analyze, and manage security-related event data from different components of an information system. SIEM platforms provide centralized visibility into security events and can support the

identification and investigation of suspicious activities.*[11]*

In cloud and server environments, security events may originate from different sources, including operating systems, applications, and other monitored components. Centralizing these events allows security information to be analyzed in a common environment rather than requiring administrators to examine individual sources separately.*[11]*

SIEM capabilities can complement cryptographic security mechanisms. Cryptographic mechanisms can verify the integrity or authenticity of data and transactions, while SIEM systems provide visibility into the events occurring within the surrounding infrastructure. This distinction is important because successful cryptographic verification alone does not provide continuous monitoring of system activity.*[11]*

Wazuh provides SIEM capabilities together with security monitoring functions. Its platform includes log analysis, intrusion and malware detection, file integrity monitoring, configuration assessment, and vulnerability detection. The Wazuh architecture includes agents deployed on monitored endpoints, a Wazuh server for analyzing received data, a Wazuh indexer for storing and indexing alerts, and a dashboard for security monitoring. *[9]*

In the proposed project, SIEM is used as an additional security-monitoring layer for the cloud storage environment. It complements the identity-based delegation, data integrity auditing, digital signatures, and trusted evidence mechanisms by providing centralized visibility into relevant security events.

2.9 Wazuh

Wazuh is an open-source security platform that provides SIEM and extended security monitoring capabilities. Its functionality includes log data analysis, intrusion and malware detection, file integrity monitoring, configuration assessment, and vulnerability detection. The platform uses agents to collect information from monitored endpoints and a central server to analyze the collected security data. *[9]*

File Integrity Monitoring (FIM) is one of the capabilities provided by Wazuh. The FIM module monitors selected files and directories and can detect file creation, modification, and deletion events. It maintains information about monitored files, including cryptographic checksums, and can generate alerts when changes are detected. Wazuh supports both scheduled and real-time monitoring depending on the configured environment. *[9]*

Wazuh can therefore provide an additional monitoring layer for cloud storage environments in which important files and system directories need to be monitored. When a monitored file is modified or deleted, the FIM mechanism can identify the change and generate an alert, providing security administrators with visibility into the event. *[9]*

In the proposed project, Wazuh is integrated with the cloud storage environment to monitor relevant security activities and file-system changes. It complements the cryptographic

mechanisms used for integrity verification rather than replacing them. The data integrity auditing mechanism provides verification of outsourced data, while Wazuh provides monitoring and alerting for relevant security events.

2.10 Nextcloud

Nextcloud is an open-source platform that provides file storage, synchronization, sharing, and collaboration capabilities in a self-hosted environment. It can be deployed on infrastructure controlled by the organization, allowing the organization to manage the storage environment and apply its own security and access policies.*[10]*

Nextcloud provides mechanisms for controlling access to files. Its File Access Control functionality allows administrators to define rules that determine whether access to files should be permitted or denied. Depending on the configured rules, access restrictions can prevent users from uploading, modifying, deleting, downloading, or synchronizing files. *[10]*

Nextcloud also provides logging capabilities that allow administrators to review system status and user and file-related activities. The platform can use its standard logging mechanism and can optionally use the Admin Audit application for additional event logging.*[10]*

These capabilities make Nextcloud suitable as the cloud storage environment for the proposed project. It provides the storage and access-management layer, while the proposed security framework adds identity-based controlled delegation, data integrity auditing, digital signatures, trusted evidence, and SIEM-based monitoring.

In the proposed implementation, Nextcloud represents the cloud storage component where the users' files are stored and managed. Its activities and relevant file-system events can be monitored by Wazuh, while the cryptographic and auditing mechanisms provide additional verification of the integrity and authenticity of the stored data.

CHAPTER TWO

3.1 Selected Work Environment

The successful implementation of any secure cloud storage framework depends largely on selecting an appropriate working environment that provides stability, flexibility, scalability, and compatibility with modern cybersecurity technologies. Since the proposed project combines cryptographic mechanisms with continuous security monitoring, the implementation environment must be capable of supporting cloud services, secure communication between multiple entities, log generation and collection, and security event analysis. Therefore, selecting an appropriate work environment represents one of the most important stages in the development of the proposed system, as it directly influences the system's performance, security, reliability, and future scalability.

Cloud computing environments have become the preferred platform for implementing modern information systems because they provide on-demand computing resources without requiring organizations to invest in expensive hardware infrastructure. These environments offer high availability, resource virtualization, flexible storage capacity, and remote accessibility, allowing users to access services from anywhere through an Internet connection. Furthermore, cloud environments support rapid deployment of applications and simplify system maintenance through centralized management, making them particularly suitable for cybersecurity applications that require continuous monitoring and rapid response to security incidents.

The proposed framework presented in this project operates in a cloud storage environment where several entities interact continuously, including the Data Owner, the Cloud Service Provider (CSP), the Trusted Third Party (TTP), and authorized users. Each participant performs specific operations such as uploading data, requesting files, verifying data integrity, exchanging cryptographic evidence, and validating digital signatures. Consequently, the implementation environment must support secure communication among all these entities while maintaining the confidentiality, integrity, and availability of exchanged information.

Another important reason for selecting a cloud-based environment is its compatibility with cryptographic security mechanisms. The proposed framework relies on hash functions to verify data integrity and digital signatures to guarantee authentication and fair non-repudiation. These cryptographic operations require an environment capable of securely executing encryption algorithms while maintaining acceptable computational performance. Modern cloud platforms provide sufficient processing power and storage resources to execute such operations efficiently without significantly affecting system performance.

In addition to cryptographic protection, modern cybersecurity strategies require continuous monitoring of system activities. Traditional security solutions generally detect attacks only after damage has already occurred, making post-incident investigation more difficult and increasing the potential impact of security breaches. For this reason, the proposed project integrates a Security Information and Event Management (SIEM) solution into the selected work environment. The SIEM platform continuously collects security logs generated by cloud servers, operating systems, applications, authentication services, and network devices. These logs are then analyzed and correlated to identify suspicious activities, detect security threats in real time, generate security alerts, and support rapid incident response.

The integration of SIEM within the selected work environment significantly enhances the overall security architecture. Instead of relying solely on cryptographic techniques to protect stored data, the system becomes capable of continuously monitoring user activities, detecting abnormal behavior, identifying potential attacks before they escalate, and maintaining detailed forensic evidence for security investigations*[1][3][4][6].*

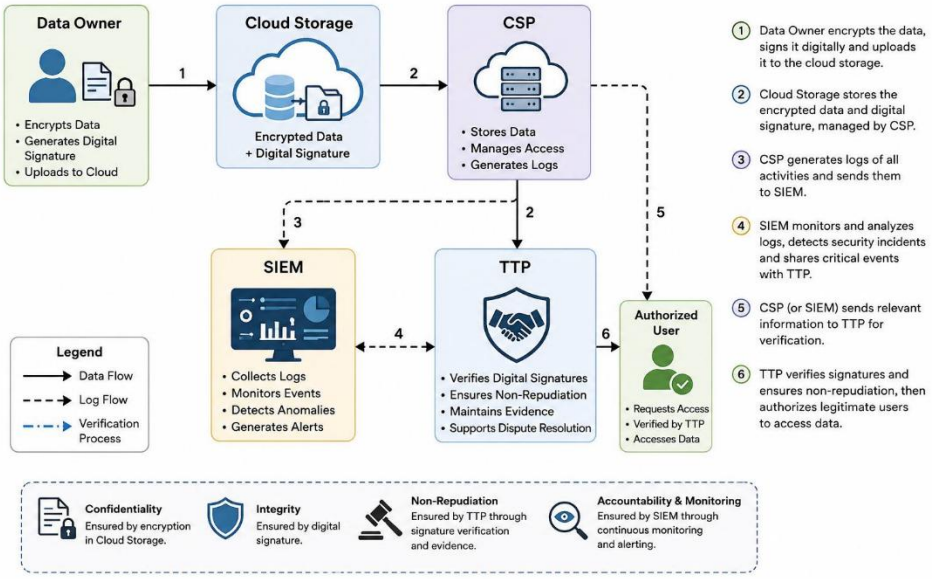


Figure 2: Proposed Work Environment

Figure 3 illustrates the overall architecture of the proposed work environment used in this project. It demonstrates the interaction between the main entities involved in the cloud storage system, including the Data Owner, Cloud Service Provider (CSP), Trusted Third Party (TTP), Authorized Users, and the integrated Security Information and Event Management (SIEM) system. The figure shows how data is securely uploaded, stored, accessed, and verified using cryptographic mechanisms, while the SIEM platform continuously collects and analyzes security logs generated by the different components. This architecture enables secure communication between all entities, ensures data integrity and fair non-repudiation, and enhances the security of the cloud environment through continuous monitoring, real-time threat detection, and rapid incident response

Component	Description
Data Owner	Uploads and manages data.
Cloud Service Provider	Stores and manages cloud resources.
Trusted Third Party (TTP)	Ensures fairness and evidence exchange.
SIEM	Collects logs, correlates events, detects threats, and generates alerts.
Authorized User	Accesses cloud data according to granted permissions.
Delegate	A user authorized by the Data Owner to upload files within a limited scope and validity period.

Table 2.1: Components of the Proposed Work Environment

3.2 Installation of the Working Environment

After selecting the appropriate work environment, the next step is to prepare and configure the software components required for implementing the proposed framework. The installation process aims to establish a stable and secure environment capable of supporting cloud storage services, cryptographic operations, and continuous security monitoring.

To achieve this objective, three main technologies were selected. VMware Workstation was used as the virtualization platform to create an isolated virtual environment without affecting the host operating system. Ubuntu Server was selected as the operating system because of its stability, security, and widespread use in server infrastructures and cybersecurity applications. Finally, Wazuh SIEM was installed to provide centralized log management, file integrity monitoring, real-time threat detection, and security event analysis.

The installation process was carried out in sequential stages to ensure compatibility between all software components. Each stage was verified before proceeding to the next one in order to guarantee a stable implementation environment for the proposed framework *[8]*

Software	Version	Purpose
VMware Workstation	Latest Stable Version	Create the virtual machine environment.
Ubuntu Server	24.04 LTS	Operating system hosting the proposed framework and Wazuh.
Wazuh	Latest Stable Version	SIEM platform for log collection, file integrity monitoring, threat detection, and security monitoring.

Table 2.2 Software Components Used in the Project

3.2.1 Installing VMware Workstation

Virtualization technology has become an essential component in modern software development, cloud computing, and cybersecurity research. Instead of relying on multiple physical computers, virtualization enables several independent operating systems to run simultaneously on a single physical machine while sharing the available hardware resources. This approach reduces hardware costs, simplifies system management, and provides an isolated environment for testing and experimentation.

In this project, VMware Workstation was selected as the virtualization platform because it provides a stable, reliable, and user-friendly environment for creating virtual machines. VMware allows researchers to simulate real server infrastructures without affecting the host operating system, making it particularly suitable for implementing cybersecurity solutions and cloud-based applications. Furthermore, VMware supports advanced features such as virtual networking, snapshot management, hardware customization, and resource allocation, all of which facilitate the implementation and testing of the proposed framework.

The installation process begins by downloading the latest stable version of VMware Workstation from the official VMware website. After downloading the installation package, the installer is executed with administrative privileges. During the installation process, the default configuration options are selected, including the installation directory, virtual network services, and system drivers required for virtualization. Once the installation is completed, the system is restarted to ensure that all virtualization services are correctly initialized.

After restarting the operating system, VMware Workstation is launched to verify that the installation has been completed successfully. At this stage, the software is ready to create virtual machines and allocate hardware resources such as processor cores, memory capacity, storage space, and network adapters. These virtual resources provide the foundation for installing Ubuntu Server and deploying the Wazuh SIEM platform in the following stages.

The selection of VMware Workstation offers several advantages for the implementation of the proposed framework. It enables complete isolation between the host and guest operating systems, supports rapid system recovery through snapshots, facilitates system testing under different configurations, and provides a flexible environment for future modifications without affecting the primary operating system. These capabilities make VMware an ideal platform for implementing cloud storage security solutions and evaluating cybersecurity mechanisms in a controlled environment.*[8]*

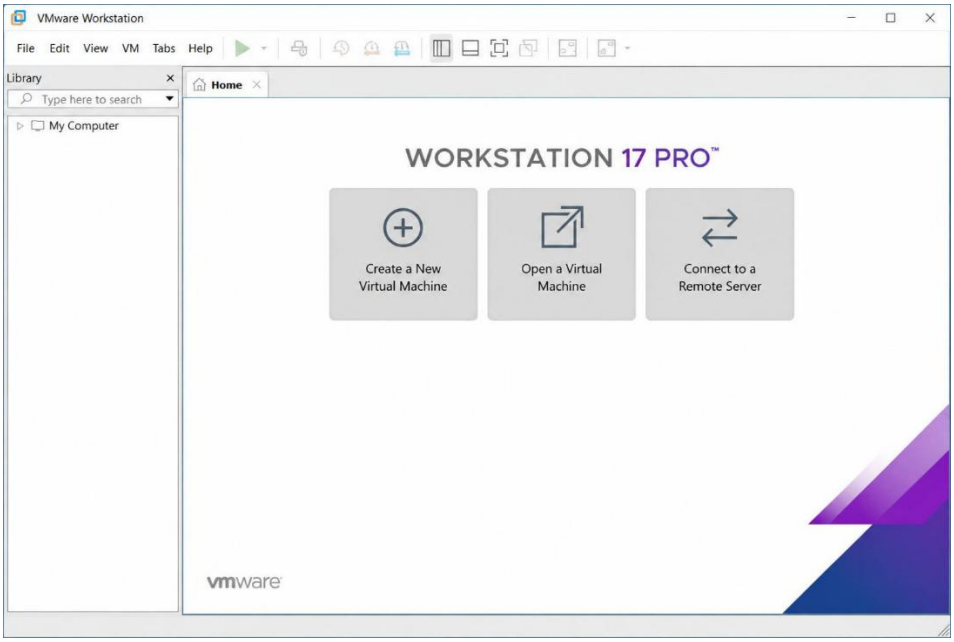


Figure 3: VMware Workstation Interface

Figure 4 shows the main interface of VMware Workstation after successful installation. The platform provides the necessary tools for creating and managing virtual machines that will host the Ubuntu Server operating system and the Wazuh SIEM platform used in this project.

3.2.2 Installing Ubuntu Server

After installing VMware Workstation, the next step is to deploy the operating system that will host the proposed framework. Ubuntu Server was selected because it is one of the most widely used Linux distributions in cloud computing, server infrastructures, and cybersecurity environments. It is well known for its stability, security, long-term support (LTS), and compatibility with a wide range of open-source security tools, making it an ideal operating system for implementing the proposed cloud storage framework.

Ubuntu Server provides a lightweight operating system optimized for server environments without unnecessary graphical components. This allows the system to utilize hardware resources more efficiently, resulting in improved performance and lower resource consumption. Moreover, Ubuntu offers regular security updates, extensive community support, and comprehensive documentation, making it suitable for both research and practical implementation.

The installation process begins by downloading the Ubuntu Server ISO image from the official Ubuntu website. A new virtual machine is then created in VMware Workstation, where the downloaded ISO image is attached as the installation media. During the virtual machine configuration, the hardware resources are allocated according to the project requirements, including processor cores, memory size, storage capacity, and network adapter configuration.

After booting from the installation media, the Ubuntu Server installation wizard guides the user through several configuration steps. These include selecting the installation language, keyboard layout, network settings, storage partitioning, server hostname, administrator username, and password. Once the configuration is completed, the operating system files are copied to the virtual disk, and the necessary system packages are installed automatically.

After the installation is completed, the virtual machine is restarted, and the administrator logs into the system using the previously configured credentials. The operating system is then updated to ensure that all installed packages are upgraded to the latest available versions. Updating the system before installing additional software is considered an important security practice because it eliminates known vulnerabilities and improves overall system stability.

Ubuntu Server serves as the foundation for the remaining stages of the project. It hosts the Wazuh SIEM platform, supports the execution of cryptographic operations, stores system logs, and provides the operating environment required for implementing and evaluating the proposed security framework. Its compatibility with cloud technologies and cybersecurity applications makes it a reliable platform for building secure cloud storage solutions *[10].*

Component	Recommended Specification
Processor	2 CPU Cores
Memory (RAM)	4 GB or more
Storage	25 GB or more
Network	NAT or Bridged Adapter
Operating System	Ubuntu Server 24.04 LTS

Table 2.3: Minimum Hardware Requirements for Ubuntu Server

Table 2.3 presents the minimum hardware requirements recommended for installing Ubuntu Server within the VMware virtual environment. These specifications were selected to ensure stable system performance while providing sufficient computing resources for running the operating system and hosting the Wazuh SIEM platform. Allocating adequate processor cores, memory, storage capacity, and network connectivity contributes to improving system reliability and enables the virtual machine to execute security monitoring, log management, and cryptographic operations efficiently throughout the implementation of the proposed framework [9]

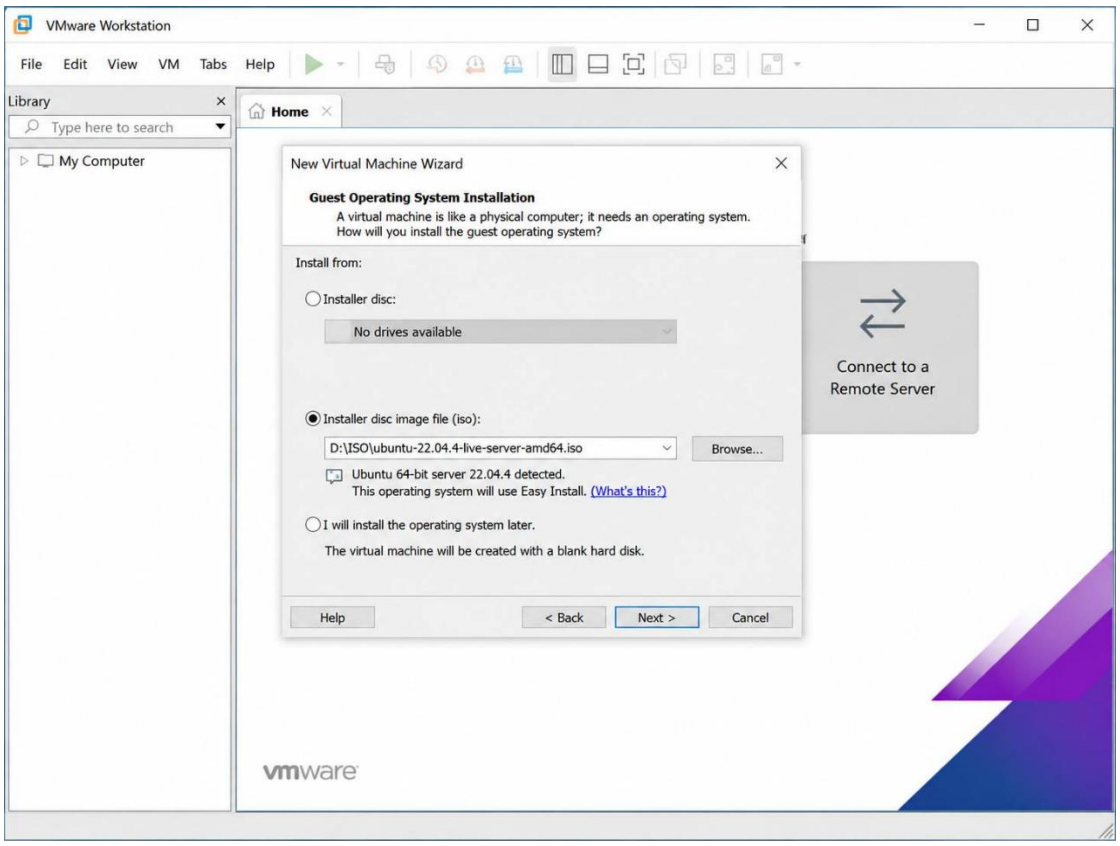


Figure 4: Ubuntu Server Installation in VMware Workstation

Figure 5 illustrates the Ubuntu Server installation process within VMware Workstation. The virtual machine is configured with the required hardware resources before installing the operating system, providing the execution environment for the proposed framework.

3.2.3 Installing Wazuh SIEM

After successfully installing Ubuntu Server, the next step is to deploy the Security Information and Event Management (SIEM) platform that will be integrated with the proposed cloud storage framework. In this project, **Wazuh** was selected as the SIEM solution because it is an open-source platform that provides comprehensive security monitoring, log management, threat detection, file integrity monitoring, vulnerability assessment, and incident response capabilities. Due to its flexibility, scalability, and compatibility with Linux-based operating systems, Wazuh has become one of the most widely adopted SIEM platforms in both academic research and enterprise cybersecurity environments.

One of the main reasons for selecting Wazuh is its ability to centralize security information collected from multiple sources. Instead of analyzing logs separately on each device, Wazuh

gathers security events from servers, operating systems, applications, and network services into a centralized platform. This centralized architecture enables security administrators to monitor the entire environment through a single dashboard, making threat detection and security analysis more efficient and organized.

The installation of Wazuh begins by preparing the Ubuntu Server environment and ensuring that the operating system has been fully updated with the latest security patches. A stable network connection is also required to download the necessary installation packages and software dependencies. Before starting the installation process, the server resources are verified to ensure that sufficient CPU, memory, and storage capacity are available for running the Wazuh services efficiently.

The deployment of Wazuh consists of several integrated components that work together to provide complete security monitoring. These components include the **Wazuh Server (Manager)**, which receives and analyzes security events, the **Wazuh Indexer**, which stores and indexes collected security data, and the **Wazuh Dashboard**, which provides a graphical interface for monitoring alerts, reviewing logs, and managing the security environment. These components cooperate to provide administrators with a centralized platform capable of processing thousands of security events generated by different systems.

Once the installation packages are downloaded, the installation process is initiated using the official deployment procedures provided by the Wazuh development team. During the installation, all required software components are automatically configured, including communication services, storage services, indexing mechanisms, and the web-based dashboard. After the installation is completed, the administrator verifies that all Wazuh services are running correctly before proceeding to the configuration stage.

After confirming the successful installation, the administrator accesses the Wazuh Dashboard through a secure web interface. This dashboard serves as the central management console for the entire security infrastructure. Through the dashboard, administrators can monitor system health, review generated alerts, analyze security events, examine system logs, manage connected agents, and investigate suspicious activities in real time. The dashboard also provides statistical reports and graphical visualizations that simplify the analysis of security incidents.

Another important capability provided by Wazuh is **File Integrity Monitoring (FIM)**. This feature continuously monitors selected files and directories for unauthorized modifications. Whenever a protected file is created, modified, deleted, or renamed, Wazuh immediately detects the change, records the event, and generates a security alert. This functionality is particularly valuable in the proposed framework because protecting the integrity of stored cloud data represents one of the primary objectives of the project.
Furthermore, Wazuh supports continuous vulnerability detection by identifying outdated software packages, missing security patches, and known system vulnerabilities.

It also performs log correlation by combining multiple security events originating from different sources in order to identify complex attack patterns that might not be detected when analyzing individual events separately. This capability significantly improves the efficiency of cybersecurity monitoring and reduces the time required to identify ongoing attacks.

The integration of Wazuh with the proposed Multi-Party Non-Repudiation (MPNR) framework represents the primary enhancement introduced in this project. While the original framework guarantees data integrity and fair non-repudiation through cryptographic techniques, Wazuh complements these mechanisms by continuously monitoring security events, detecting suspicious behavior, and generating immediate alerts whenever abnormal activities occur. Consequently, the proposed framework becomes capable of not only protecting data cryptographically but also monitoring the operational environment proactively, thereby providing an additional layer of cybersecurity protection against both internal and external threats *[11].*

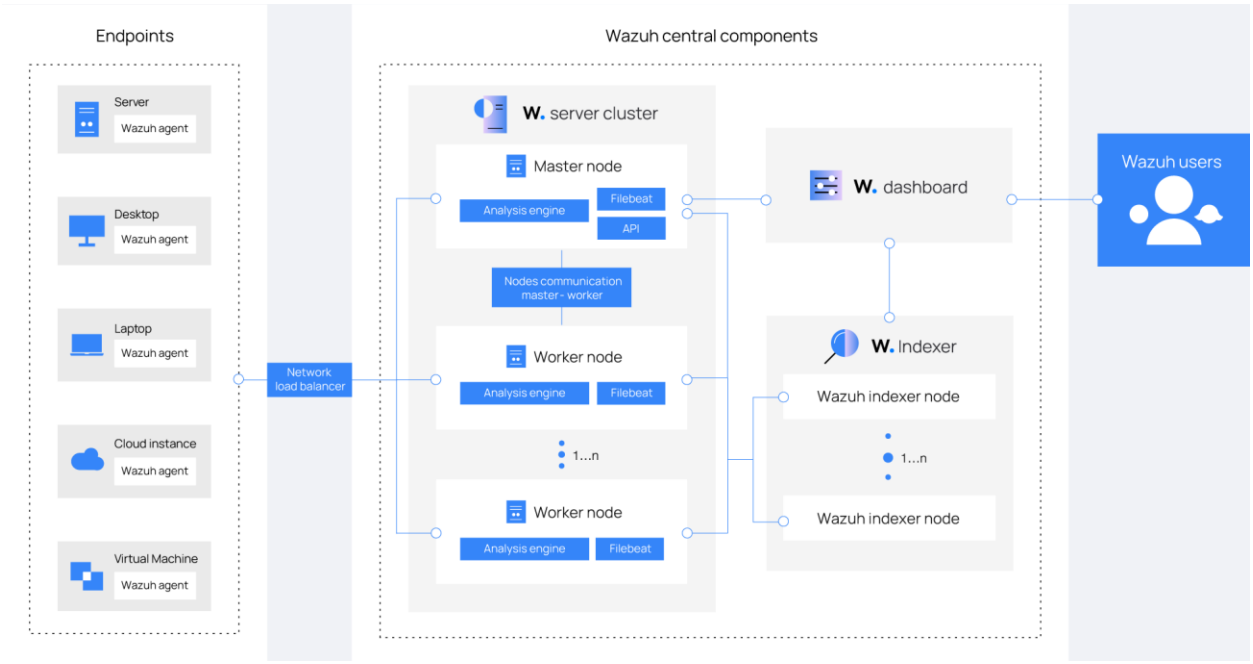


Figure 5: Wazuh SIEM Architecture

Figure 6 illustrates the architecture of the Wazuh SIEM platform used in the proposed framework. The Wazuh Manager receives security events from the monitored environment, processes and analyzes them, stores the collected data within the Wazuh Indexer, and presents security alerts and monitoring information through the Wazuh Dashboard.

3.2.4 Configuring the Environment

After completing the installation of VMware Workstation, Ubuntu Server, and the Wazuh SIEM

platform, the next stage involves configuring the working environment to ensure that all system components operate correctly and communicate efficiently. Proper configuration is essential for achieving a stable, secure, and reliable implementation environment capable of supporting the proposed cloud storage security framework.

The first step in the configuration process is to verify the network connectivity between all virtual components. The Ubuntu Server virtual machine must be connected to the virtual network provided by VMware Workstation to allow communication with the Wazuh services and enable future integration with cloud storage components. Appropriate network settings, such as assigning IP addresses and configuring the network adapter, are applied to ensure uninterrupted communication between the system components.

The operating system is then updated by installing the latest software packages and security patches available through the Ubuntu package manager. Keeping the operating system updated is considered a fundamental cybersecurity practice because it reduces the possibility of exploiting known software vulnerabilities and improves the stability and performance of the entire system.

Following the operating system configuration, the Wazuh platform is configured to initialize its core services. The Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard are verified to ensure that they are functioning correctly and communicating with each other. Any required service configurations are applied to guarantee the proper collection, processing, indexing, and visualization of security events generated within the system.

Another important configuration step involves enabling File Integrity Monitoring (FIM). Specific directories and files that are considered critical to the cloud storage framework are selected for continuous monitoring. Once configured, Wazuh continuously observes these files and automatically records any modification, deletion, creation, or unauthorized access attempt. This feature directly supports the primary objective of the proposed framework by providing continuous verification of data integrity.

Security monitoring policies are also configured during this stage. These policies define the types of security events that should be monitored, the severity levels assigned to different events, and the alert generation rules. Proper configuration of these policies enables the SIEM platform to distinguish between normal system activities and suspicious behaviors that may indicate security incidents.

Finally, the overall environment is reviewed to ensure that all software components are operating together without conflicts. Communication between the operating system, virtualization platform, Wazuh services, and future cloud storage application is verified before proceeding to the implementation stage. Completing this configuration process establishes a secure and fully operational environment capable of supporting the proposed Multi-Party Non-Repudiation (MPNR) framework with integrated SIEM functionality *[1][3][11].*

3.2.5 Verifying the Installation

After completing the installation and configuration of VMware Workstation, Ubuntu Server, and the Wazuh SIEM platform, a verification process was performed to ensure that all software components were installed successfully and operating correctly. This stage is essential because it confirms the stability of the working environment before implementing the proposed cloud storage security framework.

The verification process began by confirming that the virtual machine created in VMware Workstation could boot successfully without any hardware or software errors. The Ubuntu Server operating system was then examined to ensure that all required system services were functioning correctly and that the latest security updates had been successfully installed. The system resources, including processor utilization, memory allocation, disk space, and network connectivity, were also reviewed to verify that they satisfied the requirements of the implementation environment.

Next, the functionality of the Wazuh SIEM platform was verified. The Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard services were checked to ensure that they were running properly and communicating with each other without interruption. Successful communication between these components indicates that security events can be collected, processed, indexed, and displayed correctly through the centralized monitoring dashboard.

To validate the effectiveness of the monitoring system, several basic security events were generated within the Ubuntu Server environment. These events included user authentication activities, file creation, file modification, and system log generation. Wazuh successfully collected these events, processed them, and displayed them in the dashboard. This confirmed that the platform was capable of monitoring system activities in real time and generating security records for further analysis.

An additional verification step was performed by testing the File Integrity Monitoring (FIM) feature. A monitored file was intentionally modified, and the system immediately detected the unauthorized change. Wazuh generated a security alert containing detailed information about the modified file, the time of the event, and the type of detected activity. This experiment demonstrated the effectiveness of the monitoring mechanism and confirmed that the platform could detect potential integrity violations before they affect the cloud storage environment.

Finally, the overall environment was evaluated to ensure that all installed software components worked together as an integrated platform. The successful communication between VMware Workstation, Ubuntu Server, and the Wazuh SIEM platform confirmed that the implementation environment was fully prepared for deploying the proposed Multi-Party Non-Repudiation (MPNR) framework. Therefore, the verification stage provides confidence that the subsequent implementation and testing phases can be performed on a stable, secure, and reliable platform *[8][10][11].*

3.3 Proposed Framework Operation

The proposed framework operates through a sequence of coordinated security processes designed to guarantee secure cloud storage, preserve data integrity, provide fair non-repudiation, and continuously monitor security events. Unlike conventional cloud storage systems that mainly focus on storing and retrieving files, the proposed framework integrates cryptographic protection mechanisms with proactive security monitoring to establish a trusted cloud environment capable of resisting both accidental data modification and malicious cyberattacks.

The framework combines the original Multi-Party Non-Repudiation (MPNR) protocol with an integrated Security Information and Event Management (SIEM) platform represented by Wazuh. While the MPNR protocol is responsible for generating cryptographic evidence through hash functions, digital signatures, and the Trusted Third Party (TTP), the SIEM platform performs continuous monitoring of security events occurring throughout the cloud environment. This integration enables the framework not only to protect data cryptographically but also to detect suspicious activities, generate security alerts, and provide comprehensive forensic evidence for future investigations. In addition, the framework includes the Controlled Delegation Authentication Protocol (CDAP), described in Section 3.4, which allows the Data Owner to authorize another user to upload files under strictly controlled conditions.

The proposed operational model consists of several consecutive phases beginning with data preparation and ending with continuous monitoring after data storage. Each phase performs a specific security function that contributes to achieving one or more information security objectives. Together, these phases establish a comprehensive security architecture capable of supporting secure cloud storage services in modern distributed environments *[1][6][11].*.

3.3.1 Phase One: Data Preparation

The operational process begins when the Data Owner selects the file intended for cloud storage. At this stage, the file still exists only within the owner's local environment and has not yet been transmitted to any external entity. Before any communication with the cloud infrastructure occurs, the framework performs a set of preliminary security operations intended to prepare the file for secure storage.

The first objective of this preparation stage is to establish a trusted representation of the original data. Since any modification to the file after this moment should be detectable, the framework treats the selected file as the reference version whose integrity must remain protected throughout its lifecycle. Consequently, all subsequent cryptographic evidence generated during later stages will be directly associated with this original version of the file.

Preparing the data before transmission significantly reduces the possibility of integrity violations during communication with external cloud services. Moreover, it establishes the foundation

required for implementing fair non-repudiation because every security operation performed later will rely on the authenticity of this original file *[1][6][11].*

3.3.2 Hash Generation

After the data owner selects the file to be uploaded, the proposed framework performs the first cryptographic operation by generating a SHA-256 hash value for the original file. This process creates a fixed-length digest that uniquely represents the file content regardless of its size. Within the proposed framework, the generated hash is not used merely as a checksum; instead, it becomes the primary integrity reference associated with the uploaded file throughout its lifecycle.

The hash generation process is executed before any communication with the cloud service provider. Performing this operation locally ensures that the integrity reference is created from the original version of the file before it leaves the owner's environment. Consequently, any unauthorized modification introduced during transmission, storage, or retrieval can later be detected by recalculating the hash and comparing it with the original stored digest.

Unlike conventional cloud storage systems that mainly rely on access control mechanisms, the proposed framework combines hash generation with digital signatures and Trusted Third Party (TTP) services. Therefore, the generated hash not only detects data modification but also becomes part of the cryptographic evidence used to support fair non-repudiation between all participating entities.

From an operational perspective, the generated hash value is securely associated with the digital signature during the following stage of the framework. This association guarantees that both integrity verification and ownership verification can be performed whenever the stored file is accessed or disputed *[12].*

3.3.3 Digital Signature Generation

Purpose

Before the original file is transmitted to the cloud storage environment, the proposed framework performs a digital signature generation process. The objective of this phase is not only to authenticate the identity of the data owner but also to establish legally and technically verifiable proof that the uploaded file originated from its legitimate owner. Unlike traditional authentication methods that verify identity only during login, digital signatures remain permanently associated with the uploaded file and continue providing evidence throughout the entire storage lifecycle.

Digital signatures therefore represent one of the fundamental components of the proposed framework because they simultaneously support authentication, integrity verification, accountability, and fair non-repudiation.

Operation

Following the generation of the SHA-256 hash value, the proposed framework encrypts the generated digest using the private RSA key belonging to the data owner. The resulting encrypted digest becomes the digital signature associated with the uploaded file.

It is important to note that the framework does not encrypt the entire file using RSA because asymmetric cryptographic algorithms are computationally expensive when processing large amounts of data. Instead, only the hash value is digitally signed. This approach significantly reduces computational overhead while maintaining the same level of integrity verification and ownership authentication.

After the signature has been successfully generated, both the original file and its corresponding digital signature become logically linked and are transmitted together during the upload process.

Security Contribution

The integration of digital signatures within the proposed framework provides several security advantages. First, it guarantees data authenticity because only the legitimate owner possesses the private signing key. Second, it preserves integrity since any modification to the original file automatically changes its SHA-256 hash, causing the digital signature verification process to fail. Third, it prevents repudiation because the owner cannot later deny generating the signature without compromising the confidentiality of the private key.

Moreover, combining digital signatures with the Trusted Third Party (TTP) enables both communicating parties to obtain cryptographic evidence capable of resolving future disputes in a fair manner *[13].*

Design Note

Unlike the original MPNR framework, the proposed framework integrates the generated digital signature with centralized security monitoring through Wazuh SIEM. Consequently, cryptographic evidence and security logs complement each other, providing stronger support for forensic investigations and fair dispute resolution.

3.3.4 Secure File Upload

The secure file upload phase represents one of the most critical stages within the proposed framework because it marks the transition of data from the owner's trusted local environment into the cloud infrastructure. During this phase, the framework ensures that the uploaded information is accompanied by sufficient cryptographic evidence capable of protecting its integrity and proving its origin throughout the entire storage lifecycle.

Unlike traditional cloud storage systems that simply transfer files to remote servers, the proposed

framework requires the original file to be associated with its SHA-256 hash value and the corresponding RSA digital signature before transmission. As a result, every uploaded file becomes cryptographically protected from the very beginning of the storage process.

This approach minimizes the possibility of unnoticed data manipulation because any unauthorized modification introduced after the upload immediately changes the calculated hash value, causing the signature verification process to fail. Consequently, the cloud server stores not only the original file but also the cryptographic information required to validate its authenticity during future retrieval operations *[14].*

3.3.5 Evidence Exchange

One of the distinctive features of the proposed framework is the use of a Trusted Third Party (TTP) to facilitate the fair exchange of cryptographic evidence among participating entities. Rather than relying solely on mutual trust between the Data Owner and the Cloud Service Provider, the framework introduces an independent trusted entity responsible for preserving evidence required to resolve future disputes.

During the upload operation, the generated digital signature, together with the associated cryptographic metadata, is securely exchanged and archived by the Trusted Third Party. This evidence may include the signature itself, the corresponding public key certificate, timestamps, transaction identifiers, and integrity verification information.

The existence of an independent evidence repository significantly strengthens the fairness of the protocol because neither communicating party can independently manipulate or remove the stored evidence after the transaction has been completed *[15].*

When an upload is performed through a delegation, the evidence archived by the TTP also includes the delegation identifier and the Data Owner's signature on the delegation, so that the authority of the uploader can be proven in case of a dispute (see Section 3.4).

3.3.6 Continuous Security Monitoring using SIEM

Following the successful upload of the digitally signed file, the proposed framework enters the continuous monitoring phase. Unlike the original MPNR framework, which mainly focuses on preserving cryptographic evidence for dispute resolution, the enhanced framework introduces a Security Information and Event Management (SIEM) platform to continuously observe security-related activities occurring within the cloud storage environment.

The SIEM platform, implemented using Wazuh, continuously collects security logs from monitored systems, analyzes security events, and correlates information received from multiple sources. Rather than waiting until a dispute occurs, the monitoring process provides real-time visibility into system activities, allowing abnormal events to be identified as soon as they occur.

This capability significantly improves the overall security posture of the proposed framework by reducing the time required to detect and investigate potential security incidents *[16].*

The Wazuh platform utilizes its File Integrity Monitoring (FIM) capability to establish a trusted baseline for monitored files. During the initial scan, Wazuh records cryptographic checksums and file attributes for protected files. Subsequent scans compare the current checksum and metadata against the stored baseline. Whenever a mismatch is detected, Wazuh automatically generates a security alert indicating that the monitored file has been modified, created, or deleted.

By maintaining this baseline and continuously verifying file integrity, the framework gains the ability to detect unauthorized modifications even if an attacker attempts to manipulate stored cloud data after the original upload operation *[17].*

The integration of Wazuh with the proposed framework extends the functionality of cryptographic protection by providing operational security monitoring. Digital signatures and hash functions verify data authenticity and integrity, whereas the SIEM platform records security events, correlates suspicious activities, and preserves detailed audit logs that support forensic investigations.

The monitoring also covers delegated operations. Events such as the creation and revocation of delegations, authorized and rejected uploads, and failed integrity checks are recorded by the delegation service and collected by Wazuh, which generates alerts when the configured detection rules are matched (see Section 3.4.5).

Consequently, the proposed framework provides two complementary layers of protection. The first layer consists of cryptographic mechanisms responsible for preventing repudiation and verifying integrity. The second layer consists of continuous security monitoring capable of detecting malicious activities before they escalate into larger security incidents *[18].*

3.3.7 File Retrieval and Integrity Verification

After the requested file is retrieved from the cloud storage environment, the proposed framework performs a comprehensive integrity verification process before granting access to the user. Rather than assuming that the stored data remains unchanged, the framework independently verifies both the authenticity and integrity of the retrieved file using cryptographic mechanisms established during the upload phase.

Initially, the retrieved file is processed using the SHA-256 hashing algorithm to generate a new message digest. This newly calculated hash value is then compared with the original hash value generated when the file was first uploaded. If both values are identical, the framework confirms that no unauthorized modification has occurred during storage or transmission *[6].*

After successful hash verification, the framework proceeds to verify the digital signature using the public RSA key associated with the original data owner. Signature verification confirms that

the retrieved file was signed by the legitimate owner and that the signature has not become invalid due to unauthorized data modification.

If the verification process succeeds, the framework concludes that the file remains authentic and has not been altered since its original upload. However, if the verification fails, the framework immediately considers the retrieved data untrustworthy and prevents further processing until the detected security incident has been investigated *[13].*

Within the enhanced framework, the integrity verification stage is also integrated with the Wazuh SIEM platform. Every verification attempt is automatically recorded as a security event, including the verification timestamp, user identity, verification result, and associated file information. In cases where integrity verification fails, Wazuh immediately generates a high-priority security alert and stores the complete event within its centralized log repository.

This additional monitoring capability significantly improves the original framework by transforming integrity verification from a passive cryptographic operation into an actively monitored security process capable of supporting incident response and digital forensic investigations.

3.3.8 Experimental Environment

To evaluate the proposed framework under a realistic operating environment, a private cloud platform was designed using widely adopted and well-documented technologies. The experimental environment consists of VMware Workstation Pro as the virtualization platform, Ubuntu Server 22.04 LTS as the server operating system, Nextcloud as the private cloud storage platform, and Wazuh SIEM as the security monitoring and event management solution. In addition, SHA-256 hashing, RSA digital signatures, and the Trusted Third Party (TTP) defined in the original Multi-Party Non-Repudiation (MPNR) framework are integrated to provide integrity verification and non-repudiation services.

The virtualization layer is implemented using VMware Workstation Pro, which enables multiple virtual machines to operate on a single physical computer. This approach provides an isolated, flexible, and reproducible environment for deploying cloud services and security applications while reducing hardware requirements. VMware is extensively used in research laboratories and educational environments because it simplifies testing, configuration, and system recovery.

Ubuntu Server 22.04 LTS serves as the operating system hosting all server-side services. It was selected due to its stability, long-term support, security updates, and broad compatibility with cloud platforms and cybersecurity applications. Ubuntu Server is widely deployed for web services, cloud computing, and enterprise server infrastructures.

Nextcloud functions as the private cloud storage platform responsible for securely storing user files and managing cloud storage operations. It provides file upload, download, synchronization,

user authentication, access control, and secure data sharing through a web-based interface. Because Nextcloud is open source and can be deployed locally, it enables complete control over stored data and server configuration, making it suitable for research environments where full administrative access is required.

To strengthen the security of the cloud platform, Wazuh SIEM is deployed alongside Nextcloud on the Ubuntu Server. Wazuh continuously monitors the operating system, collects and analyzes security logs, performs File Integrity Monitoring (FIM), detects unauthorized modifications, and generates real-time alerts whenever abnormal activities are identified. This continuous monitoring complements the cryptographic mechanisms implemented by the proposed framework.

Finally, the cryptographic layer is composed of SHA-256 for generating message digests, RSA digital signatures for authentication and non-repudiation, and the Trusted Third Party (TTP) for preserving cryptographic evidence. Together, these components establish a secure and reliable cloud storage environment capable of protecting data integrity while providing continuous operational monitoring *[19].*

3.3.9 Functional Validation of the Proposed Environment

After confirming the successful deployment of the experimental environment, a functional validation was conducted to verify that all system components operate correctly as an integrated cloud storage framework. Unlike security evaluation scenarios that focus on attack simulation, this validation demonstrates the normal operational workflow of the proposed framework under standard conditions.

The validation begins with user authentication through the Nextcloud platform, followed by secure file upload, cryptographic processing using SHA-256 and RSA digital signatures, evidence preservation by the Trusted Third Party (TTP), continuous monitoring using Wazuh SIEM, and finally secure file retrieval with integrity verification. Each stage was selected according to the operational sequence defined in the original Multi-Party Non-Repudiation (MPNR) framework while incorporating the additional monitoring capabilities provided by Wazuh SIEM.

The objective of this validation is to demonstrate that all software components cooperate successfully to provide secure cloud storage services while maintaining data integrity, authentication, accountability, and non-repudiation throughout the complete file lifecycle.

3.3.10 User Authentication

The functional validation begins with user authentication through the Nextcloud private cloud platform. The data owner accesses the web interface by providing a valid username and password that have been previously configured by the system administrator. After successful

authentication, Nextcloud establishes a secure user session and grants access to the user's personal cloud storage space.

Authentication represents the first security layer of the proposed environment because it prevents unauthorized users from accessing cloud resources. At the same time, every successful login event is recorded within the operating system logs and the Nextcloud application logs. Wazuh continuously monitors these logs, allowing administrators to maintain centralized visibility of authentication activities occurring within the cloud environment.

In this implementation, the Wazuh Manager was configured to monitor the Nextcloud application log (nextcloud.log) in addition to Ubuntu system logs, enabling real-time detection of user login, file upload, file deletion, and other cloud storage activities.

Successful completion of this step confirms that the private cloud platform is functioning correctly and that authorized users are able to access cloud storage services before performing any file operations *[20].*

I. Secure File Upload

After successful authentication, the data owner uploads the required file to the private cloud storage platform through the Nextcloud web interface. Nextcloud manages file uploads using the WebDAV protocol, which provides a standardized mechanism for transferring files between clients and cloud servers while preserving authentication and access control mechanisms.

When the upload request is received, Nextcloud first authenticates the user's session and verifies the user's access permissions. If the user has sufficient privileges, the file is transferred to the server and securely stored within the Nextcloud data directory. During this process, Nextcloud records important metadata associated with the uploaded file, including the file name, owner, upload timestamp, file size, storage location, and access permissions. This metadata enables efficient file management and supports subsequent integrity verification processes.

Within the proposed framework, the file upload process represents the transition point between user interaction and cryptographic protection. Before the uploaded file becomes available for normal cloud storage operations, it is forwarded to the cryptographic processing stage, where integrity and authenticity mechanisms are applied. Therefore, although Nextcloud is responsible for providing reliable cloud storage services, the proposed framework extends its functionality by integrating cryptographic verification before the storage process is considered complete.

Successful completion of this stage confirms that the private cloud platform correctly accepts user files, stores them securely, and prepares them for the subsequent cryptographic operations defined by the proposed framework *[19][20][21].*

II. Hash Generation

The functional validation continues by generating a SHA-256 hash value for the file before it is uploaded to the Nextcloud private cloud platform. The hash generation process transforms the entire file content into a fixed-length 256-bit message digest that uniquely represents the data regardless of the original file size. After the hash value is generated, it is used as the input for the RSA digital signature process. Once the cryptographic operations have been successfully completed, the signed file is uploaded to the Nextcloud cloud storage platform for secure storage and subsequent monitoring by Wazuh SIEM.

The generated hash value serves as the digital fingerprint of the uploaded file. Even a single-bit modification in the file content produces a completely different hash value, making SHA-256 an effective mechanism for detecting unauthorized data modification. Unlike encryption algorithms, SHA-256 is a one-way cryptographic hash function, meaning that the original file cannot be reconstructed from the generated hash value.

Within the proposed framework, the calculated SHA-256 digest becomes the fundamental integrity reference for all subsequent security operations. The generated digest is forwarded to the digital signature module, where it will be signed using the RSA private key. At the same time, the digest is prepared for storage as cryptographic evidence by the Trusted Third Party (TTP), ensuring that the original integrity information remains available whenever future verification is required.

Integrating SHA-256 into the upload workflow provides an additional protection layer beyond traditional cloud storage services. While Nextcloud securely stores the uploaded file, the proposed framework independently preserves a cryptographic representation of the file content, enabling reliable integrity verification throughout the complete file lifecycle *[19][21].*

The SHA-256 hash was generated experimentally using OpenSSL on the Ubuntu Server.

III. Digital Signature Generation

Once the SHA-256 message digest has been generated, the proposed framework immediately performs the digital signature operation. Instead of signing the original uploaded file, the RSA private key is applied to the generated SHA-256 digest. This approach significantly reduces computational complexity while maintaining the same level of integrity protection and authentication.

The generated digital signature uniquely binds the identity of the data owner to the uploaded file. Since only the legitimate owner possesses the private signing key, the resulting signature provides cryptographic proof that the uploaded file originated from the authenticated user. Any subsequent modification to the file content automatically

invalidates the original signature because the recalculated SHA-256 digest will no longer match the signed digest.

Within the proposed framework, the digital signature becomes one of the primary pieces of cryptographic evidence supporting the non-repudiation process. Together with the corresponding SHA-256 digest and upload timestamp, the signature is forwarded to the Trusted Third Party (TTP), where it is securely preserved for future verification and dispute resolution if required.

By integrating RSA digital signatures immediately after the upload process, the proposed framework extends the traditional storage functionality provided by Nextcloud with strong cryptographic authentication. Consequently, the uploaded file is not only securely stored within the cloud platform but is also associated with verifiable cryptographic evidence confirming its authenticity and ownership.
RSA key generation, signing, and signature verification were performed using OpenSSL.

IV. Cryptographic Evidence Preservation by the Trusted Third Party (TTP)

Following the successful generation of the digital signature, the proposed framework transfers all cryptographic evidence to the Trusted Third Party (TTP). This stage represents one of the most important phases of the framework because it preserves trustworthy evidence that can later be used to verify data ownership and resolve possible disputes between participating entities.

The information delivered to the TTP includes the SHA-256 message digest, the RSA digital signature, the timestamp, and the identity of the authenticated data owner. These elements are securely stored without modifying the original uploaded file maintained by the Nextcloud platform. Instead, the TTP preserves independent cryptographic evidence that can be retrieved whenever authenticity verification becomes necessary.

The separation between cloud storage and evidence preservation improves the reliability of the proposed framework. While Nextcloud is responsible for storing and managing user files, the TTP maintains the cryptographic records required for integrity verification and non-repudiation. Consequently, even if the cloud storage system experiences accidental failures or administrative modifications, the original cryptographic evidence remains available through the Trusted Third Party.

At the completion of this stage, both the uploaded file and its associated cryptographic evidence have been successfully preserved. This establishes a trusted reference point that will later be used during the integrity verification process after the file is retrieved from the cloud storage environment *[21].*

V. Continuous Monitoring Using Wazuh SIEM

After the uploaded file and its corresponding cryptographic evidence have been successfully stored, the Wazuh SIEM platform begins continuous monitoring of the cloud environment. Unlike the cryptographic mechanisms that operate only during upload and verification, Wazuh remains active throughout the entire file lifecycle by continuously observing security events generated within the Ubuntu Server and the Nextcloud platform.

The monitoring process starts by collecting security logs from both the operating system and the Nextcloud application. These logs contain valuable information regarding user authentication, file upload activities, access attempts, permission changes, and administrative operations. Wazuh centralizes these logs and continuously analyzes them to provide complete visibility of all activities occurring inside the cloud storage environment.

In addition to log collection, Wazuh performs File Integrity Monitoring (FIM) on the Nextcloud data directory. During the initial monitoring phase, Wazuh establishes a trusted baseline by recording the original attributes of each monitored file, including file size, permissions, ownership, timestamps, and cryptographic checksums. This baseline becomes the reference point used to evaluate every subsequent file operation.

Whenever a monitored file is modified, renamed, deleted, or replaced, Wazuh automatically compares the current file state with the previously established baseline. If any unexpected difference is detected, the platform immediately records the event and generates a real-time security alert for the system administrator. This continuous monitoring capability enables rapid identification of unauthorized changes while complementing the cryptographic protection mechanisms provided by SHA-256 and RSA digital signatures.

Unlike the original MPNR framework, which verifies integrity only when cryptographic validation is requested, integrating Wazuh introduces proactive security monitoring that continuously supervises the cloud environment. Consequently, the proposed framework not only guarantees integrity verification and non-repudiation but also provides early detection of suspicious activities before they evolve into significant security incidents.*[21]*

VI. Security Event Logging and Alert Generation

Following continuous monitoring of the Nextcloud environment, Wazuh automatically records all observed security events within its centralized logging system. Every monitored activity, including user authentication, file uploads, file downloads, file modifications, permission changes, and system events, is collected and stored for subsequent analysis. This centralized logging mechanism provides complete visibility of the operational status of the proposed cloud environment.

Whenever Wazuh identifies an event that matches predefined monitoring rules, the platform automatically generates a security alert. The generated alert contains detailed information regarding the detected event, including the event type, affected file, timestamp, system user, severity level, and rule identifier. These alerts are immediately displayed through the Wazuh Dashboard, allowing system administrators to monitor cloud activities in real time.

The alert generation mechanism does not modify the uploaded file or interfere with the cryptographic operations performed by the proposed framework. Instead, it complements the integrity verification process by providing immediate operational awareness whenever important events occur inside the cloud platform. This enables administrators to investigate abnormal activities without waiting until the file verification stage.

By integrating centralized logging with real-time alert generation, the proposed framework extends *[11]*

VII. Secure File Retrieval and Integrity Verification

The final stage of the functional validation begins when an authorized user requests to download a previously stored file from the Nextcloud private cloud platform. After successful authentication, Nextcloud retrieves the requested file from its storage directory and securely delivers it to the user. Before the download process is considered complete, the proposed framework performs a comprehensive integrity verification procedure to ensure that the retrieved file remains authentic and unchanged.

First, the framework recalculates the SHA-256 hash value of the downloaded file using the same hashing algorithm applied during the upload stage. The newly generated digest is then compared with the original SHA-256 digest securely preserved by the Trusted Third Party (TTP). If both digest values are identical, the framework confirms that the file content has remained unchanged throughout its storage period.

Following successful hash verification, the framework validates the RSA digital signature associated with the file. Using the corresponding public key, the stored signature is verified against the recalculated message digest. Successful signature verification confirms that the uploaded file originated from the legitimate data owner and that no unauthorized modifications have occurred since the original upload operation.

To validate the effectiveness of the integrity mechanism, the file was intentionally modified after signing. Recalculating the SHA-256 hash produced a different digest, and signature verification returned “Verification failure”, confirming that unauthorized modifications are successfully detected.

Throughout this verification process, Wazuh continues monitoring the cloud environment

by recording download activities and generating security logs associated with file access operations. Although Wazuh does not participate directly in cryptographic verification, its continuous monitoring capabilities provide an additional operational security layer by documenting every retrieval event performed within the cloud platform.

Successful completion of hash comparison, digital signature verification, and operational monitoring confirms that the proposed framework preserves data integrity, authentication, accountability, and non-repudiation throughout the complete lifecycle of the stored file. Consequently, the retrieved file can be considered trustworthy and identical to the original version uploaded by the authorized data owner *[21]*.

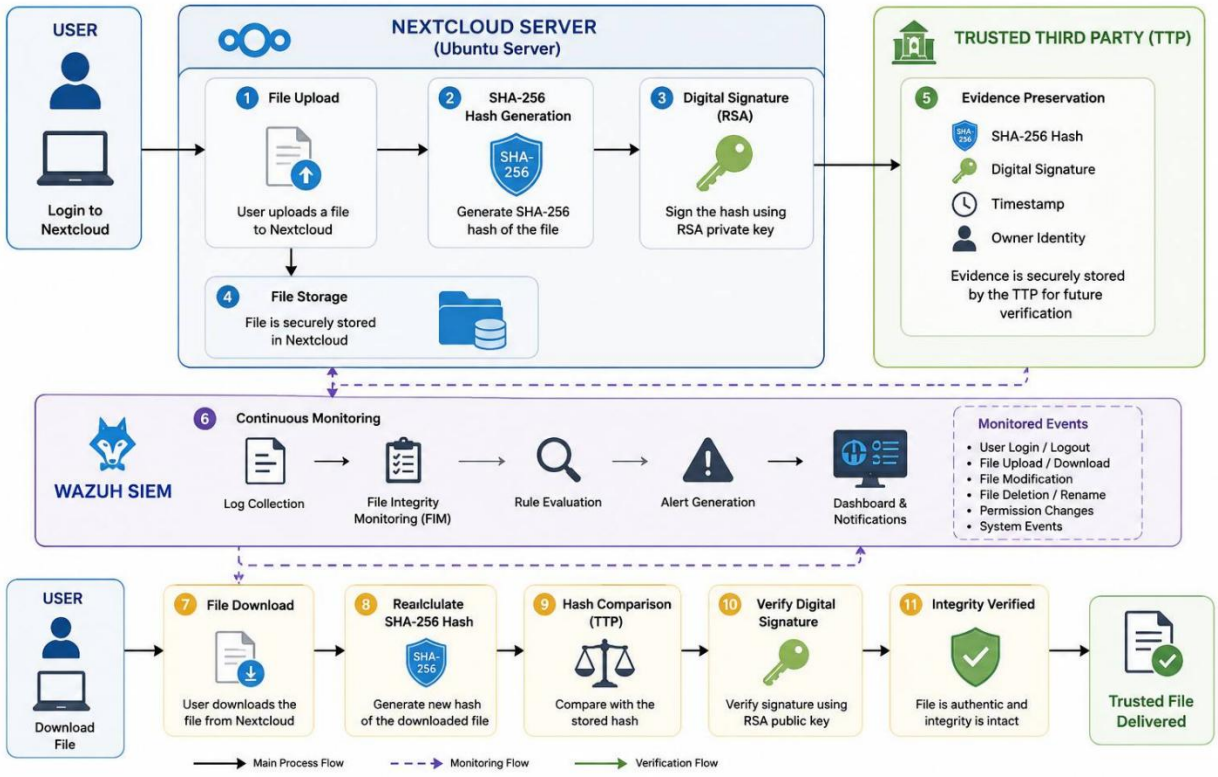


Figure 6: Overall Workflow of the Proposed Cloud Storage Security Framework

Figure 7 illustrates the complete operational workflow of the proposed cloud storage security framework. The process begins with user authentication through the Nextcloud platform, followed by secure file upload. The uploaded file is processed by generating a SHA-256 hash and an RSA digital signature. The original file is stored in Nextcloud, while the cryptographic evidence, including the hash value, digital signature, timestamp, and user identity, is securely preserved by the Trusted Third Party (TTP). Meanwhile, Wazuh SIEM continuously monitors the cloud environment by collecting security logs, performing File Integrity Monitoring (FIM),

and generating alerts when suspicious activities are detected. Finally, during file retrieval, the framework recalculates the SHA-256 hash and verifies the RSA digital signature to confirm the integrity and authenticity of the downloaded file Figure 7 shows the workflow in which the Data Owner uploads files directly. The delegated upload workflow, in which an authorized delegate uploads on behalf of the owner, is described in Section 3.4.

3.4 Proposed Controlled Delegation Authentication Protocol (CDAP)

The framework described in Section 3.3 assumes that the Data Owner personally uploads every file to the cloud. In practice, an owner often needs another user to upload files on their behalf, for example a team member submitting project documents. Allowing this without control would expose the cloud to impersonation, privilege abuse, and replayed requests. To address this, the project introduces the Controlled Delegation Authentication Protocol (CDAP), which allows a Data Owner to authorize another user to upload files to the cloud while guaranteeing identity verification, restricted permissions, limited validity, and full documentation of every important operation. CDAP builds on the identity-based controlled delegation concept of the reference scheme [2] and connects it with the digital signature, Trusted Third Party (TTP), and SIEM mechanisms already used in the proposed framework.
The protocol consists of two phases: **Phase One** creates and registers the delegation, and **Phase Two** uses the delegation to upload a file.

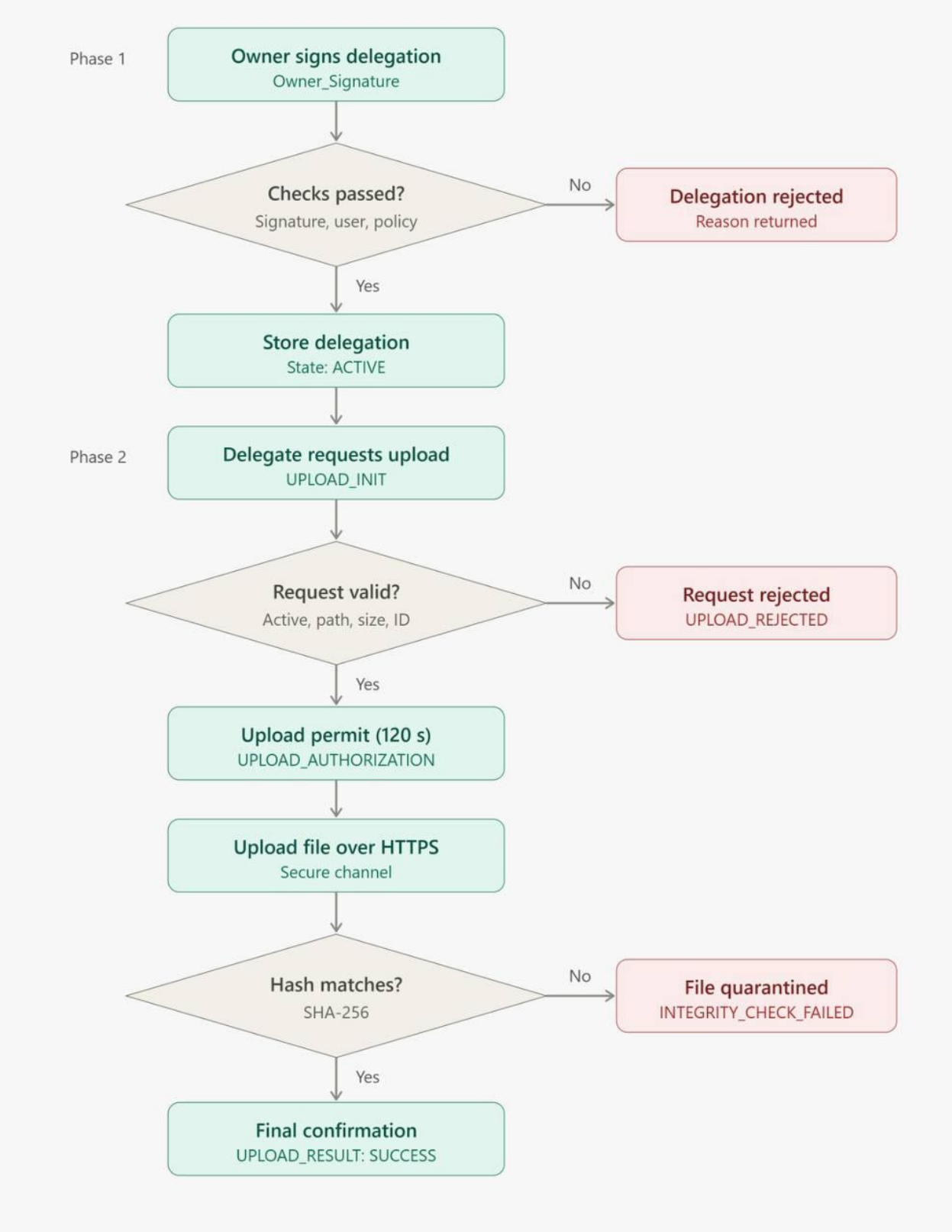


Figure 7: CDAP workflow showing both phases and the rejection paths

3.4.1 Entities and Assumptions

Entity	Role in CDAP
Data Owner (User A)	Creates the delegation and signs it with their private key. Can revoke it at any time.
Delegate (User B)	Authenticates to the cloud with their own account and uploads files within the delegated scope.
Cloud Service Provider (CSP)	Hosts the delegation service, verifies delegations and upload requests, issues temporary upload permits, and stores files.
Trusted Third Party (TTP)	Preserves the cryptographic evidence of delegated uploads, as in Section 3.3.5.
Wazuh SIEM	Collects the security events generated by the delegation service and raises alerts.

Table 3.1: Entities Involved in CDAP and Their Roles

The protocol assumes that each user owns an RSA key pair, that the public keys are bound to user identities through trusted certificates, and that all communication takes place over a secure channel such as HTTPS. SHA-256 is used for hashing and RSA for digital signatures, consistent with Section 3.3.3. The authorization decision is made by the delegation service on the cloud server; Wazuh does not approve individual messages.

3.4.2 Phase One: Creating the Delegation

Step 1: The owner submits a delegation request. The Data Owner logs in to the system and specifies the identity of the delegate, the folder to which uploads are allowed, the permission type (UPLOAD only), the start and end dates of the delegation, and any additional restrictions such as the maximum file size. The system then builds a delegation message containing this information.

Step 2: The owner signs the delegation. The system computes a SHA-256 digest of the delegation message, and the owner signs it with their RSA private key. An example message is shown below.

```
Delegation_ID:   DEL-1001
Owner_ID:        User_A
Delegate_ID:     User_B
Cloud_ID:        Cloud_01
Allowed_Action:  UPLOAD
Allowed_Path:    /Project/
Valid_From:      2026-11-02 10:00
Valid_Until:     2026-11-09 18:00
Max_File_Size:   10 MB
Owner_Signature: [Digital Signature]
```

The signature is essential because it allows the cloud to verify that the delegation was issued by the owner whose public key is used for verification, provided that the identity bound to that key

has been properly certified. It also prevents the owner from later denying that the delegation was issued.

Step 3: The cloud verifies the delegation. The owner sends the signed message to the cloud, which checks:

1. The validity of the digital signature.
2. The identity of the delegating owner.
3. The existence of the delegate in the system.
4. That the end date of the delegation is later than its start date.
5. That the requested permissions are allowed.
6. That there is no conflict with the cloud's policies.

If all checks succeed, the cloud stores the delegation and sets its state to ACTIVE. Otherwise, the delegation is rejected and the reason for rejection is returned.

Step 4: The cloud confirms registration. The cloud sends a response to the owner:

```
Message_Type:     DELEGATION_RESPONSE
Delegation_ID:    DEL-1001
Status:           ACCEPTED
Delegation_State: ACTIVE
Server_Timestamp: 2026-11-02 10:05
```

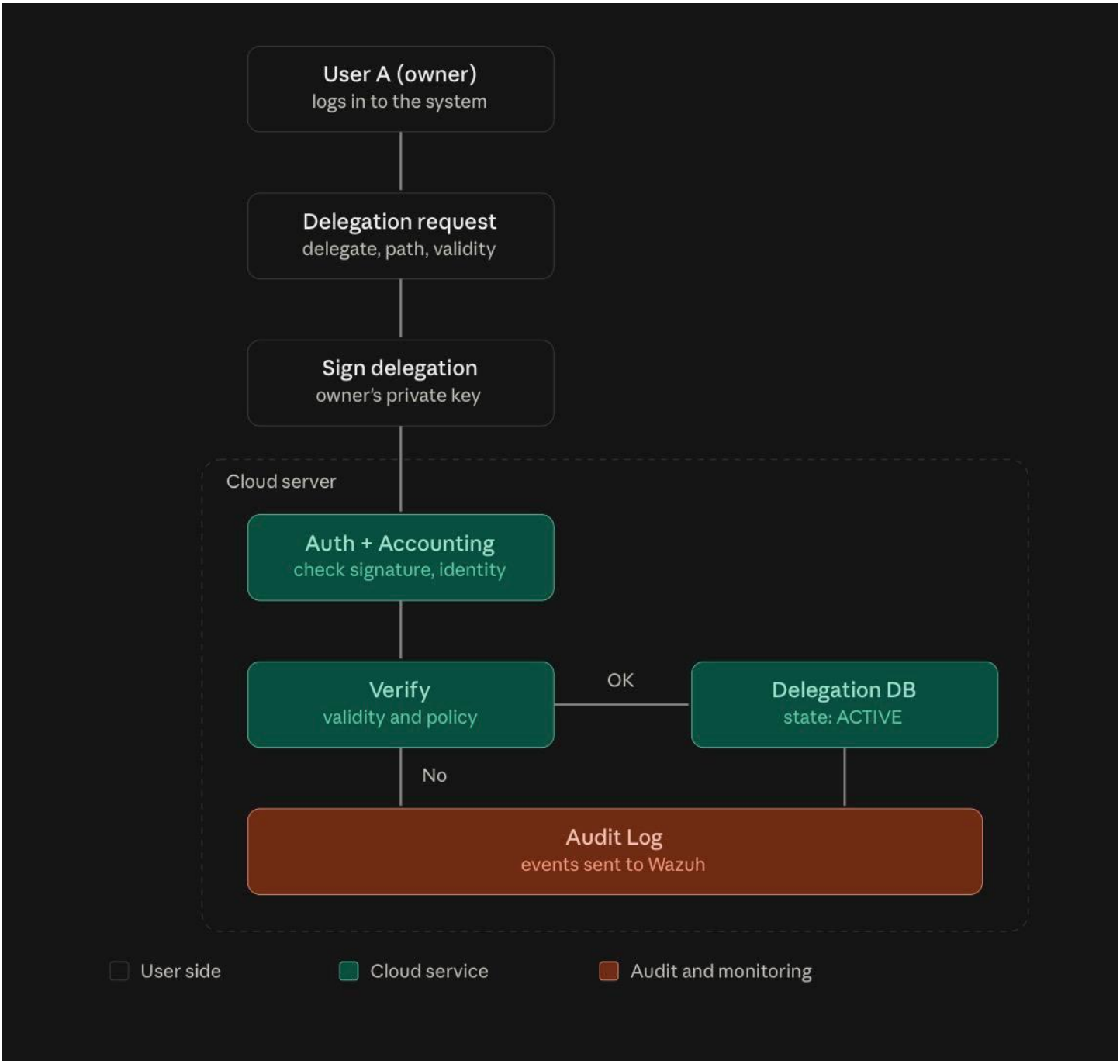


Figure 8: Phase One – creating and registering a delegation

3.4.3 Phase Two: Uploading a File Using the Delegation

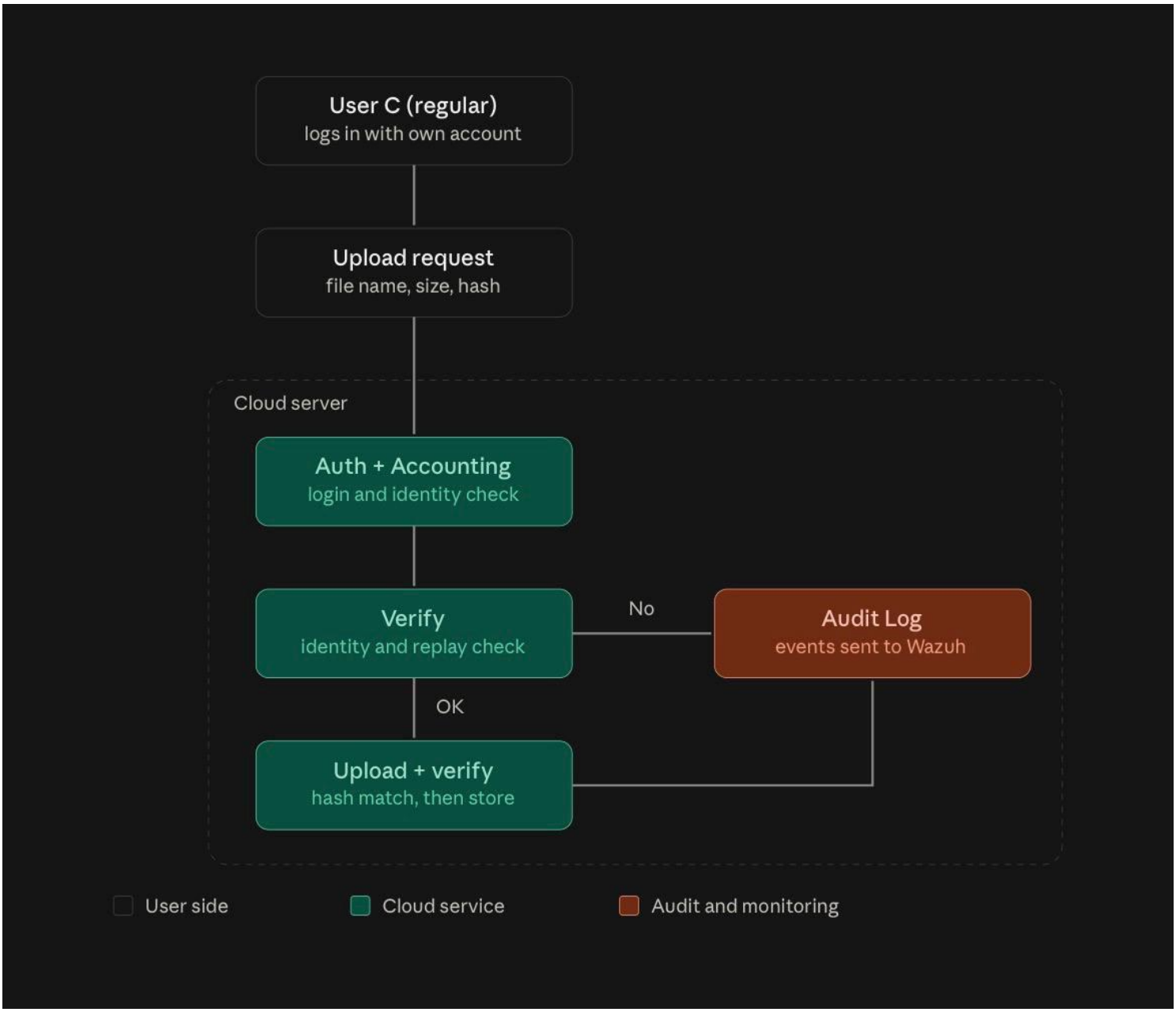


Figure 9: Standard upload by a regular user without delegation

Step 1: The delegate requests the start of an upload. After logging in with their own account, the delegate computes the SHA-256 hash of the file, signs it with their own private key, and sends an upload initiation request:

```
Message_Type:       UPLOAD_INIT
Delegate_ID:        User_B
Delegation_ID:      DEL-1001
File_Name:          report.pdf
File_Size:          2 MB
File_Hash:          SHA256(file)
Request_ID:         REQ-2001
Delegate_Signature: [Digital Signature]
```

The file hash helps verify the integrity of the content, but by itself it neither proves the identity of the uploader nor prevents a previous request from being reused. For this reason the request carries the delegate's signature and a unique Request_ID.

Step 2: The cloud validates the request. The cloud verifies that:

1. The logged-in user is actually User B and the delegate signature is valid.
2. The delegation DEL-1001 exists and is still ACTIVE.
3. The delegation was issued by the correct file owner.
4. The requested operation is UPLOAD.
5. The requested path is covered by the delegation.
6. The file size is within the allowed limit.
7. The Request_ID is new and has not been used before.

If the delegation is expired, revoked, or does not cover the requested path, the operation is rejected.

Step 3: The cloud issues a temporary upload permit. Instead of giving the delegate unrestricted capabilities, the cloud issues a permit that is valid only for the current upload:

```
Message_Type:    UPLOAD_AUTHORIZATION
Request_ID:      REQ-2001
Delegation_ID:   DEL-1001
Upload_ID:       UP-3001
Allowed_Action:  UPLOAD
Allowed_Path:    /Project/report.pdf
Expires_In:      120 seconds
```

Step 4: The delegate uploads the file. The delegate sends the file over a secure channel such as HTTPS. The cloud checks that the upload is linked to the valid permit and then receives and stores the file.

Step 5: The cloud verifies file integrity. After receiving the file, the cloud computes the SHA-256 hash of the content it actually received and compares it with the hash sent by the delegate at the start of the operation. If the two hashes match, the received content is identical to the content that was hashed before upload. If they differ, the file is rejected or quarantined and the event is logged. The comparison must be performed on the final received content, not merely on the hash supplied by the user.

Step 6: The cloud sends a final confirmation.

```
Message_Type:     UPLOAD_RESULT
Upload_ID:        UP-3001
Delegation_ID:    DEL-1001
Delegate_ID:      User_B
File_Name:        report.pdf
Final_Hash:       SHA256(file)
Integrity_Status: VERIFIED
Upload_Status:    SUCCESS
Server_Timestamp: 2026-11-02 10:07
```

Step 7: Evidence preservation. After a successful upload, the cloud forwards the evidence of the operation to the TTP, as described in Section 3.3.5. This evidence includes the final SHA-256 hash, the delegate's signature, the Delegation_ID together with the owner's signature on the delegation, the timestamp, and the Upload_ID. As a result, the TTP can later prove both that the file was uploaded by the delegate and that the delegate was authorized by the owner at that time.

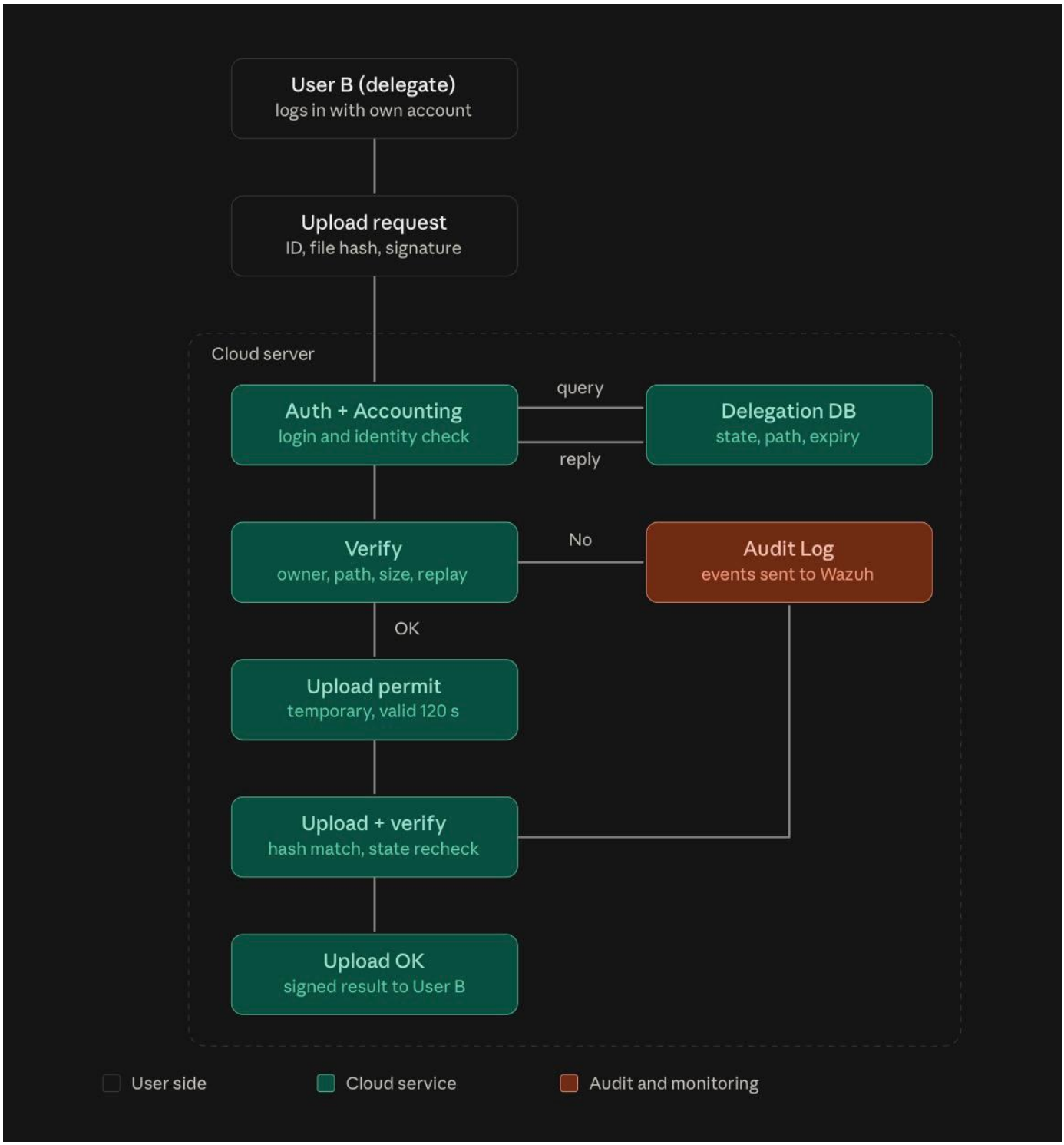


Figure 10: Phase Two – delegated upload by User B

3.4.4 Revoking a Delegation

The owner may cancel a delegation at any time by sending a signed revocation request:

```
Message_Type:    DELEGATION_REVOKE
Delegation_ID:   DEL-1001
Owner_ID:        User_A
Owner_Signature: [Digital Signature]
```

The cloud verifies the identity of the owner and the validity of the signature, changes the delegation state to REVOKED, and sends a confirmation. From that moment, any new upload request using this delegation must be rejected. In addition, the server must check the delegation state when the upload is actually executed, and not only when the request is initiated. This prevents a delegate from exploiting an old permit after the delegation has been revoked.

3.4.5 Integration with Wazuh SIEM

Wazuh does not need to approve every message exchanged between the user and the cloud, and the authorization decision remains inside the delegation service on the server. Instead, the delegation service writes the following security events to a log file, which the Wazuh agent collects in the same way as the Nextcloud application log described in Section 3.3.10:

Event	Generated when
DELEGATION_CREATED	A delegation is verified and set to ACTIVE.
DELEGATION_REVOKED	The owner revokes a delegation.
UPLOAD_AUTHORIZED	An upload permit is issued.
UPLOAD_REJECTED	An upload request fails validation.
INTEGRITY_CHECK_FAILED	The received file hash does not match the declared hash.
UPLOAD_COMPLETED	An upload finishes successfully and is verified.

Table 3.2: Security Events Generated by the CDAP Delegation Service

Wazuh collects these events and raises alerts whenever the detection rules configured for the environment are matched, for example repeated UPLOAD_REJECTED events from the same delegate or an INTEGRITY_CHECK_FAILED event.

3.4.6 Security Analysis of CDAP

Threat	Countermeasure
Forged delegation	The delegation is signed by the owner and the signature is verified by the cloud.
Impersonation of the delegate	Authenticated login together with the delegate's signature on the upload request.
Privilege abuse	Delegation is limited to the UPLOAD action, a specific path, a maximum file size, and a validity period.
Replay of an old request	Unique Request_ID and a short-lived upload permit bound to one Upload_ID.
Use of a delegation after revocation	The delegation state is checked at the moment the upload is executed.
Content modification during upload	The cloud recomputes the SHA-256 hash of the received content and compares it with the declared hash.
Denial of the delegation or the upload	Owner and delegate signatures are preserved as evidence by the TTP.
Undetected abuse	Security events are logged and analysed by Wazuh.

Table 3.3: Security Threats and Countermeasures in CDAP

Chapter Three

Practical Implementation

4.1 Experimental Environment

The objective of this chapter is
to implement the proposed Multi-Party Non-Repudiation (MPNR) framework presented in the research paper within a real cloud storage environment. Unlike a theoretical analysis, this chapter focuses on transforming the proposed security mechanisms into a practical implementation capable of protecting cloud data integrity and providing non-repudiation services.

To achieve this objective, a complete experimental environment was prepared using virtualization technology. VMware Workstation Pro 17 was selected to create a dedicated Ubuntu Server virtual machine that represents the cloud server. Ubuntu Server 22.04 LTS was chosen because it is a stable, secure, and widely used operating system for cloud computing environments. In addition, Apache Web Server and MariaDB Database Server were installed to host the cloud storage platform.

Nextcloud was selected as the cloud storage application because it provides an open-source cloud environment supporting file uploading, downloading, and user management. Therefore, it can accurately represent the Cloud Service Provider (CSP) assumed by the proposed MPNR protocol.

To implement the cryptographic operations required by the protocol, OpenSSL was installed and used to generate RSA public/private key pairs and perform digital signature operations. The generated digital signatures provide authentication, integrity verification, and non-repudiation services for uploaded files.

Finally, Wazuh SIEM was integrated into the environment to monitor every security-related operation. Wazuh continuously collects security logs, analyzes system events, and generates real-time alerts whenever suspicious activities or protocol violations are detected. This monitoring component allows the proposed implementation to simulate a real security operation center capable of detecting attacks against cloud storage services.

4.2 Cloud Storage Deployment

The first stage of the practical implementation focused on establishing a secure cloud storage environment capable of supporting the proposed Multi-Party Non-Repudiation (MPNR) framework. According to the research paper, all security mechanisms are built upon the existence of a cloud storage platform where the data owner uploads files and authorized users later download them. Therefore, creating a realistic cloud environment was considered an essential step before implementing the proposed security protocol.

To achieve this objective, Ubuntu Server 22.04 LTS was installed on a VMware virtual machine. After the operating system was configured, Apache Web Server and MariaDB Database Server were installed to provide the required web and database services. Apache acts as the web service responsible for hosting the cloud application, whereas MariaDB stores all user accounts, file

metadata, and cloud configuration information.

Once the web server and database were configured successfully, the Nextcloud platform was installed. Nextcloud was selected because it is one of the most widely used open-source cloud storage platforms, providing secure file storage, user authentication, access control, and web-based file management. The installation process included creating the database, configuring the web interface, and initializing the administrator account.

After completing the installation, the Nextcloud dashboard was successfully accessed through a web browser. This confirmed that the cloud storage environment was fully operational and ready to receive uploaded files. The successful deployment of Nextcloud provided a practical representation of the Cloud Service Provider (CSP) described in the proposed MPNR framework.

The implemented cloud environment allows the owner to upload files, while authorized users can later access the stored data through the web interface. This behavior closely matches the cloud storage model discussed in the research paper and provides the foundation for implementing the remaining security mechanisms, including digital signatures, broadcast sessions, non-repudiation receipts, trusted third-party services, and integrity verification.

Furthermore, using a real cloud storage platform instead of a simulated environment increases the credibility of the experimental results. It demonstrates that the proposed MPNR protocol can be integrated into existing cloud storage systems without requiring modifications to the underlying storage architecture. Consequently, the proposed implementation shows that the security framework can be practically deployed using widely available open-source technologies.

Practical Results

The cloud deployment process was completed successfully, and the following components were verified:

Ubuntu Server was running correctly.
Apache Web Server was operational.
MariaDB Database Server was successfully connected.
Nextcloud was accessible through the web browser.
User authentication and file management functions were operating normally.

The successful completion of this experiment confirms that the experimental cloud storage environment satisfies all infrastructure requirements needed for implementing the proposed Multi-Party Non-Repudiation protocol.

4.3 Digital Signature Implementation

One of the fundamental security mechanisms proposed in the MPNR framework is the use of digital signatures to guarantee data authenticity, integrity, and non-repudiation. Before a file is uploaded to the cloud storage platform, the owner digitally signs the file using a private cryptographic key. Later, every receiver verifies the signature using the corresponding public key before accepting the downloaded data.

In this implementation, the OpenSSL cryptographic library was used to generate an RSA public/private key pair. The private key remained only with the data owner and was used exclusively for signing files. The public key was distributed to authorized users, allowing them to verify the authenticity of the received files without exposing the owner's private key.

Initially, an RSA key pair was generated using OpenSSL. After that, a sample file named **message.txt** was created to represent the cloud data. The owner then generated a digital signature (**message.sig**) using the private key. During the verification process, OpenSSL compared the received file with the digital signature using the owner's public key.

To automate this process, a Bash script named **verify_signature.sh** was developed. The script performs the following tasks automatically:

Reads the original file (message.txt).
Reads the digital signature (message.sig).
Uses the owner's public key (public_key.pem).
Verifies the integrity and authenticity of the file.
Records the verification result into the system log.

If the verification succeeds, the script generates the message:

Digital Signature Verification SUCCESS

If the file has been modified or the signature is invalid, the script generates:

Digital Signature Verification FAILURE

These log messages are automatically monitored by the Wazuh SIEM platform, allowing security administrators to detect any unauthorized modification immediately.

The implementation demonstrates that any change to the file contents invalidates the digital signature. Consequently, even a single modified character causes the verification process to fail. This behavior confirms that the proposed mechanism effectively protects cloud data integrity and prevents attackers from replacing or modifying stored files without being detected.

Moreover, integrating digital signatures into the cloud storage environment provides non-repudiation. Since only the owner's private key can generate a valid signature, the owner cannot later deny uploading the original file. Similarly, users can independently verify the authenticity

of the received data using the corresponding public key.

Practical Results

The digital signature mechanism was successfully implemented and tested.

The experimental results showed two possible scenarios:

Case 1 – Valid File

The file remained unchanged.
Signature verification succeeded.
OpenSSL displayed **Verified OK**.
Wazuh generated a **Digital Signature Verification SUCCESS** alert.

Case 2 – Modified File

The file content was intentionally modified.
Signature verification failed.
OpenSSL displayed **Verification Failure**.
Wazuh generated a **Digital Signature Verification FAILURE** alert.

These results confirm that the implemented digital signature mechanism successfully detects any unauthorized modification of cloud data and provides strong authentication and non-repudiation services, as required by the proposed MPNR framework.

4.4 Trusted Third Party (TTP) and Resolve Mode

One of the major contributions of the proposed Multi-Party Non-Repudiation (MPNR) framework is the introduction of a Trusted Third Party (TTP) responsible for resolving disputes between the participating entities. In traditional cloud storage systems, when a disagreement occurs between the data owner, cloud provider, or user, there is often no trusted mechanism capable of determining which party is responsible. The proposed MPNR protocol addresses this limitation by introducing a Resolve Mode that preserves all available digital evidence whenever a verification failure occurs.

According to the research paper, the Trusted Third Party does not participate in every communication session. Instead, it becomes active only when a dispute occurs or when one of the protocol participants fails to complete the transaction successfully. This design minimizes computational overhead while maintaining fairness among all participants.

To simulate the behavior of the Trusted Third Party, a dedicated directory named **TTP** was created inside the Ubuntu environment. Within this directory, a subfolder named **evidence** was implemented to store all digital evidence generated during the protocol execution.

Whenever the verification process detects an invalid digital signature or modified file contents, the Resolve Mode is automatically activated. During this process, a Bash script named **resolve_mode.sh** collects all available evidence and stores it inside the TTP directory.

The stored evidence includes:

The original message file.
The digital signature.
The owner's identification.
The original file hash.
The resolution timestamp.

By preserving these files, the Trusted Third Party can later reconstruct the complete transaction history and determine which participant violated the protocol. Consequently, neither the owner nor the receiver can falsely deny the performed operation because the collected evidence remains permanently available for forensic investigation.

The Resolve Mode was fully automated during the implementation. Whenever the digital signature verification fails, the verification script immediately calls the Resolve Mode without requiring manual intervention. This behavior closely follows the protocol proposed in the research paper, where dispute resolution is automatically initiated after detecting protocol violations.

The implemented TTP significantly improves fairness within the cloud storage environment. Instead of relying solely on trust between communicating parties, all important evidence is securely preserved for future verification. This mechanism strengthens accountability and reduces the possibility of false claims during dispute resolution.

Practical Results

The Resolve Mode was successfully tested by intentionally modifying the original file before executing the verification process.

After detecting the modified file, the system automatically performed the following operations:

Signature verification failed.
Resolve Mode was activated.
Digital evidence was copied into the TTP directory.
A timestamp was generated.
The owner information was preserved.
The file hash was stored for later investigation.

The evidence directory successfully contained:

message.txt

message.sig
hash.txt
owner.txt
resolve_time.txt

These experimental results demonstrate that the implemented Trusted Third Party successfully preserves digital evidence whenever protocol violations occur. Therefore, the proposed implementation satisfies the fairness and non-repudiation requirements described in the original MPNR framework.

4.5 Broadcast Session Implementation

One of the most significant contributions introduced by the proposed Multi-Party Non-Repudiation (MPNR) framework is the Broadcast Session. Unlike conventional cloud storage systems, where users completely trust the cloud provider during file downloads, the proposed protocol establishes a direct communication channel between the data owner and authorized users.

The primary objective of the Broadcast Session is to prevent rollback attacks. In a rollback attack, the cloud provider may intentionally or unintentionally provide an outdated version of a file instead of the most recent one. Since users cannot distinguish between the old and new versions by themselves, such attacks may remain undetected in traditional cloud storage systems.

According to the proposed protocol, immediately after uploading a file to the cloud storage platform, the data owner broadcasts the cryptographic hash value of the uploaded file to all authorized users. This broadcast message is transmitted independently of the cloud provider, ensuring that the provider cannot manipulate or replace the distributed integrity information.

To implement this mechanism, the SHA-256 hash value of the uploaded file was generated using OpenSSL tools. The generated hash was stored in a file named **owner_hash.txt**, representing the original integrity value calculated by the data owner.

The broadcast operation was simulated by securely copying the **owner_hash.txt** file into the authorized user's directory. This process represents the independent broadcast communication channel proposed in the original MPNR protocol.

On the receiver side, a Bash script named **verify_owner_hash.sh** was developed to verify the received hash. The script calculates a new SHA-256 hash from the downloaded file and compares it with the broadcast hash received directly from the owner.

If both hash values are identical, the verification process reports:

Broadcast Hash Verification SUCCESS

However, if the hash values differ, the script immediately reports:

Broadcast Hash Verification FAILURE

A hash mismatch indicates that the downloaded file differs from the version originally uploaded by the owner. Such behavior represents a rollback attack or unauthorized data modification performed by the cloud provider or another attacker.

To improve security monitoring, the generated verification messages were forwarded to the Wazuh SIEM platform. Dedicated detection rules were created to generate security alerts whenever a successful or failed broadcast verification occurred.

This implementation demonstrates how the Broadcast Session establishes a trusted communication path between the owner and users without relying entirely on the cloud provider. Consequently, users become capable of independently verifying that they received the latest version of the uploaded file.

Practical Results

The Broadcast Session was successfully implemented and tested using two different scenarios.

In the first scenario, the original file remained unchanged after being downloaded. The calculated SHA-256 hash matched the owner's broadcast hash, resulting in a successful verification. Wazuh generated a **Broadcast Hash Verification SUCCESS** alert, confirming that the downloaded file was authentic and up to date.

In the second scenario, the downloaded file was intentionally modified to simulate a rollback attack. The calculated hash no longer matched the owner's broadcast hash. Consequently, the verification process failed, and Wazuh generated a **Broadcast Hash Verification FAILURE** alert indicating a potential rollback attack.

The successful implementation of the Broadcast Session confirms the effectiveness of the proposed MPNR protocol in protecting cloud users against rollback attacks. By independently distributing the original file hash, the owner enables users to verify data freshness without depending solely on the cloud storage provider.

4.6 Non-Repudiation Receipt (NRR) Implementation

Non-Repudiation of Receipt (NRR) is one of the core security services provided by the proposed Multi-Party Non-Repudiation (MPNR) framework. Its primary objective is to provide undeniable evidence that the receiver has successfully received the transmitted file. Without this mechanism, a malicious receiver could falsely claim that the file was never received, resulting in disputes between the communicating parties.

According to the proposed protocol, after successfully receiving the uploaded file, the receiver generates a receipt confirming the completion of the download process. This receipt is digitally signed by the receiver and returned to the data owner. Consequently, both the sender and the receiver possess cryptographic evidence proving that the file transmission has been completed successfully.

To simulate this mechanism, a receipt file named **NRR_receipt.txt** was created to represent the acknowledgement message generated by the receiver. The receipt contains information identifying the completed transaction and confirms that the receiver successfully obtained the transmitted file.

To protect the receipt from forgery or modification, the generated receipt was digitally signed, producing the file **NRR_receipt.sig**. The digital signature guarantees that the receipt originated from the legitimate receiver and prevents unauthorized parties from creating fake acknowledgements.

A verification script named **verify_nrr.sh** was developed to automate the validation process. The script verifies the integrity and authenticity of the received receipt before it is accepted by the owner.

When the verification succeeds, the following message is generated:

NRR Verification SUCCESS

If the receipt or its digital signature has been modified, the verification immediately fails and generates:

NRR Verification FAILURE

These verification messages are continuously monitored by the Wazuh SIEM platform. Dedicated detection rules were configured to generate security alerts whenever receipt verification succeeds or fails.

The implemented NRR mechanism significantly improves fairness within the communication process. Since both parties possess digitally signed evidence, neither the sender nor the receiver can later deny their participation in the transaction. Therefore, the implemented mechanism successfully satisfies the non-repudiation requirement proposed by the MPNR framework.

Practical Results

The Non-Repudiation Receipt mechanism was successfully implemented and experimentally verified.

During the first experiment, the receipt remained unchanged after being generated. The verification process successfully validated the digital signature, and Wazuh generated an **NRR Verification SUCCESS** alert confirming the validity of the receipt.

In the second experiment, the receipt file was intentionally modified to simulate an attempted forgery. The verification process immediately detected the modification, resulting in an **NRR Verification FAILURE** message. Wazuh subsequently generated a security alert indicating that the receipt evidence had been tampered with.

These results demonstrate that the implemented NRR mechanism effectively protects receipt authenticity and prevents receivers from denying successful file reception. Therefore, the proposed implementation successfully fulfills the non-repudiation objectives described in the original MPNR protocol.

4.7 Download Verification

After completing the upload, broadcast, and non-repudiation phases, the final step of the proposed Multi-Party Non-Repudiation (MPNR) framework is to verify the downloaded file before it is accepted by the receiver. According to the proposed protocol, users should never trust the downloaded file directly from the cloud provider. Instead, the file must pass multiple security verification procedures to ensure its authenticity, integrity, and freshness.

To implement this requirement, a dedicated verification script named **download_verify.sh** was developed. The script performs a complete security verification immediately after the file is downloaded from the cloud storage platform.

The verification process consists of two independent security checks.

The first verification checks the integrity of the downloaded file. A new SHA-256 hash value is calculated from the downloaded file and compared with the original hash value received from the owner during the Broadcast Session. If both hash values are identical, the file integrity is confirmed. Otherwise, the verification process immediately reports that the downloaded file has been modified or replaced.

The second verification validates the digital signature attached to the downloaded file. Using the owner's public key, OpenSSL verifies that the digital signature was generated by the legitimate owner and that the file contents have not changed since the signature was created.

Only when both verification procedures succeed is the downloaded file considered trustworthy. If either verification fails, the file is rejected and a security event is generated.

To automate security monitoring, the verification script records its result in the system log. Wazuh continuously monitors these log entries and generates corresponding alerts.

When both integrity verification and digital signature verification succeed, the script generates the following message:

Download Verification SUCCESS

However, if either verification fails, the script reports:

Download Verification FAILURE

This implementation significantly improves cloud storage security because users no longer rely solely on the cloud provider. Instead, every downloaded file is independently verified using cryptographic evidence generated by the owner.

Furthermore, combining broadcast verification with digital signature verification provides stronger protection than either mechanism individually. While the broadcast hash confirms that the latest file version has been received, the digital signature guarantees authenticity and non-repudiation. Together, these mechanisms establish a highly reliable verification process consistent with the objectives of the proposed MPNR framework.

Practical Results

The Download Verification mechanism was successfully implemented and evaluated under different scenarios.

During the first experiment, the downloaded file remained unchanged after being transferred from the cloud storage server. The calculated SHA-256 hash matched the owner's broadcast hash, and the digital signature verification succeeded. Consequently, the verification process reported **Download Verification SUCCESS**, and Wazuh generated a corresponding security alert confirming that the downloaded file was authentic and trustworthy.

In the second experiment, the downloaded file was intentionally modified before executing the verification script. The calculated hash no longer matched the owner's original hash, and the digital signature verification also failed. As a result, the script generated **Download Verification FAILURE**, and Wazuh immediately reported the incident through a high-priority security alert.

These experimental results demonstrate that the implemented download verification mechanism effectively protects cloud users from receiving modified or unauthorized files. Therefore, the proposed implementation successfully satisfies the integrity verification requirements described in the original MPNR framework.

4.8 Rollback Attack Simulation

One of the most serious threats discussed in the original research paper is the **Rollback Attack**. In this type of attack, the cloud service provider returns an old version of a stored file instead of the latest version without informing the user. Since the file may still contain a valid digital signature, the user may incorrectly assume that the downloaded file is the newest version.

The authors of the proposed Multi-Party Non-Repudiation (MPNR) framework identified this vulnerability as one of the major weaknesses of traditional cloud storage systems. To overcome this problem, they proposed the Broadcast Session, in which the owner distributes the latest hash value directly to all authorized users. This allows users to independently verify that the downloaded file matches the latest version uploaded by the owner.

To validate this solution experimentally, a rollback attack was simulated in the implemented cloud environment. After the original file had been successfully uploaded and verified, its contents were intentionally modified to represent an outdated or tampered version. The modified file was then treated as if it had been downloaded from the cloud storage server.

The verification process was executed using the developed verification scripts. First, the SHA-256 hash of the downloaded file was compared with the original hash value received from the Broadcast Session. Since the modified file no longer matched the owner's hash, the integrity verification failed immediately.

Next, the digital signature verification was performed using the owner's public key. Because the file contents had changed after the signature was generated, OpenSSL reported a verification failure. This confirmed that the downloaded file was no longer authentic.

Following the verification failure, the implemented Resolve Mode was activated automatically. The Trusted Third Party (TTP) collected and stored all available digital evidence, including the modified file, its digital signature, the owner's identity, the original hash value, and the timestamp of the incident.

Simultaneously, the generated security logs were analyzed by the Wazuh SIEM platform. Wazuh detected the failed verification process and generated real-time alerts indicating that the downloaded file had failed integrity verification and that a possible rollback or tampering attack had occurred.

The experiment demonstrates that the proposed MPNR framework can successfully detect rollback attacks before the modified file is accepted by the user. Instead of blindly trusting the cloud provider, the user independently verifies both the broadcast hash and the digital signature, ensuring that only the latest authentic version of the file is accepted.

Practical Results

The rollback attack simulation produced the following results:

The downloaded file was intentionally modified.
The SHA-256 hash comparison failed.
Digital signature verification failed.
The Resolve Mode was activated automatically.
Digital evidence was preserved inside the TTP directory.
Wazuh generated security alerts indicating the verification failure.

These experimental results confirm that the implemented Broadcast Session and Resolve Mode effectively detect rollback attacks and preserve sufficient digital evidence for future dispute resolution. Therefore, the proposed implementation successfully satisfies one of the primary security objectives of the original MPNR framework.

4.9 Wazuh SIEM Monitoring and Alert Generation

Security monitoring is an essential component of modern cloud computing environments. While cryptographic mechanisms such as digital signatures, hash functions, and non-repudiation protocols provide strong protection against unauthorized modifications, security administrators still require a centralized monitoring system capable of collecting security events, analyzing suspicious activities, and generating real-time alerts. For this reason, the Wazuh Security Information and Event Management (SIEM) platform was integrated into the proposed implementation.

In this work, Wazuh was deployed on the Ubuntu Server environment together with the cloud storage platform. The deployment included the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard. The Manager was responsible for collecting security logs, the Indexer stored the collected events, and the Dashboard provided a graphical interface for monitoring alerts and security incidents.

After successfully installing the Wazuh components, the local system log file **/var/log/digital_signature.log** was connected to Wazuh through the **ossec.conf** configuration file. This configuration enabled Wazuh to continuously monitor the log file and analyze every security event generated by the developed Bash scripts.

To classify different security events, custom detection rules were implemented inside the **local_rules.xml** file. Each rule was assigned a unique identifier and a severity level corresponding to the importance of the detected event. Whenever one of the implemented verification scripts recorded a security message inside the system log, Wazuh immediately matched the event against the corresponding rule and generated a real-time alert.

The developed rules covered all implemented security mechanisms. Digital signature verification events generated alerts indicating successful or failed signature validation. Broadcast Session verification generated alerts identifying successful integrity verification or potential rollback attacks. The Non-Repudiation Receipt (NRR) mechanism generated alerts confirming receipt validation or detecting receipt tampering. Finally, the download verification process generated alerts confirming successful file verification or reporting integrity failures.

By integrating all security mechanisms into a centralized monitoring platform, administrators could observe every stage of the proposed protocol through a single dashboard. This significantly improved system visibility and reduced the time required to identify suspicious activities.

Practical Results

The Wazuh SIEM platform successfully detected and classified all generated security events during the experimental evaluation.

Whenever the verification scripts executed successfully, Wazuh generated informational alerts indicating successful protocol execution. Conversely, whenever intentional file modifications or protocol violations were introduced, Wazuh immediately generated high-priority alerts identifying the detected security incident.

4.10 Overall Experimental Scenario

To evaluate the effectiveness of the proposed Multi-Party Non-Repudiation (MPNR) framework, a complete end-to-end experimental scenario was implemented. The objective of this experiment was to simulate the entire life cycle of a secure cloud storage transaction, beginning with file creation and ending with dispute resolution. The experiment combined all previously implemented security mechanisms into one integrated workflow, demonstrating how they cooperate to provide authenticity, integrity, fairness, and non-repudiation.

The experiment started when the data owner prepared the file intended for cloud storage. Before uploading the file, an RSA digital signature was generated using the owner's private key. This signature served as cryptographic proof of the file's authenticity and guaranteed that any future modification could be detected during the verification process.

After signing the file, it was uploaded to the Nextcloud platform, which represented the Cloud Service Provider (CSP). The successful upload confirmed that the cloud storage environment was ready to distribute the protected file to authorized users.

Immediately after the upload operation, the owner calculated the SHA-256 hash of the uploaded file. This hash was distributed independently to authorized users through the Broadcast Session. Since the broadcast channel operates separately from the cloud provider, users received a trusted integrity value directly from the owner without depending on the cloud storage service.

Next, the receiver downloaded the file from the Nextcloud server. Before accepting the downloaded file, the receiver executed the developed verification scripts. The downloaded file hash was first compared with the broadcast hash received from the owner. After confirming the integrity of the file, the digital signature was verified using the owner's public key. Only after both verification procedures succeeded was the downloaded file considered authentic and accepted by the receiver.

Following successful verification, the receiver generated a Non-Repudiation Receipt (NRR), confirming that the file had been received successfully. The receipt was digitally signed and later verified to ensure its authenticity. This provided undeniable evidence that the communication process had been completed successfully and prevented the receiver from denying file reception.

To demonstrate the security capabilities of the proposed framework, a rollback attack was then simulated by intentionally modifying the stored file before executing the verification process again. As expected, the hash comparison failed, the digital signature verification also failed, and the verification scripts rejected the modified file.

Immediately after the verification failure, the Resolve Mode was activated automatically. The Trusted Third Party (TTP) collected all available digital evidence, including the modified file, digital signature, owner information, original hash value, and the timestamp of the detected incident. This preserved all necessary forensic evidence required for future dispute resolution.

Throughout every stage of the experiment, the Wazuh SIEM platform continuously monitored the generated security logs. Each successful or failed verification triggered the corresponding detection rule, resulting in real-time alerts displayed on the Wazuh Dashboard. Consequently, administrators were able to observe the complete execution of the protocol and immediately identify any abnormal behavior.

The successful execution of this integrated experimental scenario confirms that the proposed implementation satisfies all major security objectives of the MPNR framework. The developed environment successfully combines cloud storage services, digital signatures, broadcast verification, non-repudiation receipts, trusted third-party evidence preservation, and centralized SIEM monitoring into a unified security solution.

Therefore, the practical implementation demonstrates that the proposed protocol can be effectively deployed using open-source technologies while providing strong protection against unauthorized modifications, rollback attacks, and dispute-related security threats.

Chapter Four

Results and Discussion

5.1 Experimental Results

The practical implementation demonstrated that the proposed Multi-Party Non-Repudiation (MPNR) framework can be successfully integrated into a cloud storage environment using open-source technologies. All proposed security mechanisms were implemented and experimentally validated, including digital signatures, Broadcast Session, Non-Repudiation Receipt (NRR), Trusted Third Party (TTP), Download Verification, and SIEM monitoring through Wazuh.

The conducted experiments confirmed that every implemented security mechanism performed successfully under normal operating conditions. Files uploaded to the cloud storage platform remained authentic and verifiable through RSA digital signatures, while SHA-256 hash verification successfully confirmed data integrity after downloading.

Furthermore, the Broadcast Session effectively distributed the original file hash to authorized users, enabling them to verify that the downloaded file represented the latest uploaded version. This mechanism successfully prevented rollback attacks that could not be detected in conventional cloud storage environments.

The Non-Repudiation Receipt mechanism generated undeniable cryptographic evidence confirming successful file reception. Consequently, neither the sender nor the receiver could later deny their participation in the communication process.

Finally, Wazuh SIEM continuously monitored all implemented verification mechanisms and successfully generated real-time alerts whenever successful or failed verification events occurred. This significantly improved the visibility of security events throughout the entire protocol execution.

5.2 Security Evaluation Before and After Applying the Proposed Framework

To evaluate the effectiveness of the proposed implementation, a comparison was performed between a conventional cloud storage environment and the implemented MPNR framework.

Security Feature	Traditional Cloud Storage	Framework
Digital Signature Verification	Not Available	Implemented
File Integrity Verification	Partial	Full SHA-256 Verification
Broadcast Hash Distribution	No	Yes
Rollback Attack Detectic	Impossible	Detected Immediately
Non-Repudiation	Not Supported	Fully Supported
Trusted Third Party	Not Available	Implemented
Automatic Evidence Collection	No	Yes
SIEM Monitoring	No	Wazuh Integration
Real-Time Alerts	No	Available

Table 5.1: Security Comparison Between Traditional Cloud Storage and the Proposed MPNR Framework

Table 5.1 clearly demonstrates that the proposed implementation provides significantly stronger security than a traditional cloud storage environment. The addition of cryptographic verification, dispute resolution, and centralized monitoring substantially enhances trust between cloud participants.

5.3 Discussion

The experimental results indicate that the proposed framework successfully satisfies the primary security objectives identified in the original research paper.

First, digital signatures effectively prevented unauthorized modification of uploaded files. Any intentional change immediately caused the signature verification process to fail.

Second, the Broadcast Session successfully addressed one of the major limitations of conventional cloud storage systems by enabling users to verify the freshness of downloaded files. Consequently, rollback attacks were detected before compromised files could be accepted.

Third, the Trusted Third Party preserved sufficient digital evidence whenever protocol violations occurred. This guarantees fairness among all participants and supports future dispute resolution.

Finally, integrating Wazuh SIEM significantly improved operational security by providing continuous monitoring and immediate detection of abnormal protocol behavior. The generated alerts enabled rapid identification of integrity violations, signature failures, rollback attacks, and receipt tampering.

Overall, the obtained results confirm that the practical implementation closely follows the security objectives proposed by the original MPNR framework while demonstrating that the protocol can be successfully deployed using widely available open-source software.

5.4 Overall Achievement of the Proposed Framework

The implemented system achieved all objectives defined at the beginning of this work.

The cloud storage environment was successfully deployed using Nextcloud and Ubuntu Server. The proposed cryptographic mechanisms were integrated into the storage workflow without modifying the underlying cloud architecture. Digital signatures, broadcast verification, non-repudiation receipts, dispute resolution through the Trusted Third Party, and centralized monitoring using Wazuh operated together as one integrated security framework.

The conducted experiments demonstrated that the proposed implementation successfully detects unauthorized file modifications, rollback attacks, and protocol violations while preserving sufficient evidence for future forensic investigations. Therefore, the developed environment provides stronger security guarantees than conventional cloud storage systems and confirms the practical applicability of the proposed Multi-Party Non-Repudiation framework.

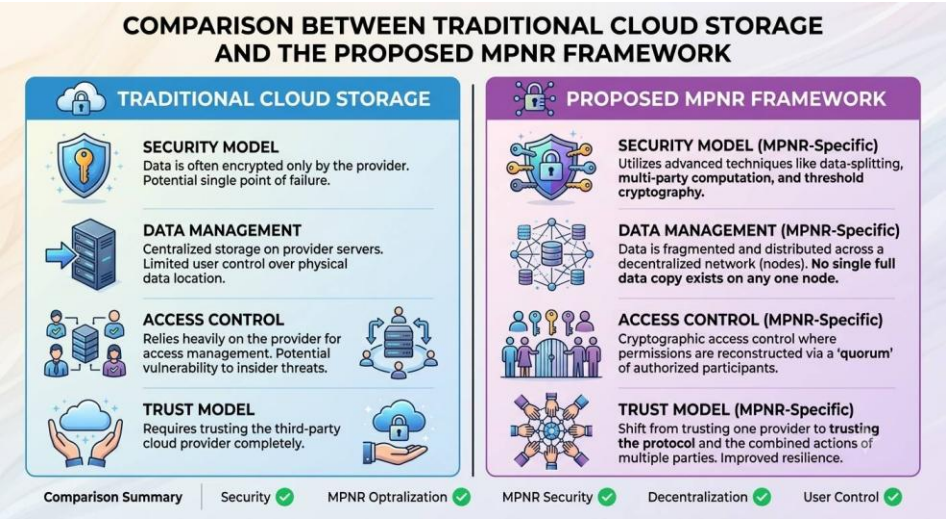


Figure 11: Comparison Between Traditional Cloud Storage and the Proposed MPNR Framework

Conclusion

This research presented the practical implementation and evaluation of a **Multi-Party Non-Repudiation (MPNR)** framework designed to enhance security and trust in cloud storage environments. The implementation successfully transformed the proposed concepts described in the original research paper into a real experimental system using open-source technologies, including Ubuntu Server, Nextcloud, OpenSSL, and Wazuh SIEM.

Throughout this work, a secure cloud storage environment was established to support the implementation of several security mechanisms. Digital signatures were employed to ensure data authenticity, integrity, and non-repudiation. A Broadcast Session was implemented to distribute the original hash value directly to authorized users, enabling them to detect rollback attacks. The Non-Repudiation Receipt (NRR) mechanism provided undeniable evidence of successful file reception, while the Trusted Third Party (TTP) preserved digital evidence whenever verification failures or disputes occurred.

In addition, the integration of the Wazuh SIEM platform provided centralized security monitoring and real-time alert generation. Custom detection rules were developed to monitor all implemented security operations, allowing immediate identification of digital signature failures, rollback attacks, download verification failures, and receipt tampering. This integration significantly improved the visibility and manageability of security events within the cloud environment.

The experimental evaluation demonstrated that the proposed framework successfully achieved its primary objectives. All implemented security mechanisms operated correctly under normal conditions and effectively detected intentional file modifications and simulated rollback attacks. Furthermore, the developed implementation confirmed that the proposed MPNR protocol can be integrated into existing cloud storage platforms without requiring significant architectural changes.

Overall, the obtained results demonstrate that combining cryptographic techniques with trusted dispute resolution mechanisms and SIEM-based security monitoring substantially improves the security, reliability, and trustworthiness of cloud storage systems. Therefore, the proposed implementation represents a practical and effective solution for protecting cloud data against unauthorized modifications, rollback attacks, and non-repudiation disputes while maintaining fairness among all participating entities.

—

Future Work

Future research may extend this work by implementing the proposed framework in a distributed multi-cloud environment involving multiple cloud service providers. Additional cryptographic techniques, such as Attribute-Based Encryption (ABE) or Blockchain technology, may also be integrated to further strengthen access control, transparency, and decentralized trust

management. Furthermore, machine learning techniques could be incorporated into the SIEM platform to improve anomaly detection and predict potential security threats before they occur. Finally, evaluating the proposed framework under large-scale cloud environments with multiple simultaneous users would provide a deeper understanding of its scalability, performance, and practical deployment in real-world cloud infrastructures.

References

[1] J. Feng and Y. Chen, "A fair non-repudiation framework for data integrity in cloud storage services," International Journal of Cloud Computing, vol. 2, no. 1, pp. 20–47, 2013, doi: 10.1504/IJCC.2013.050954.

[2] P. Goswami, N. Faujdar, S. Debnath, A. K. Khan, and G. Singh, "Investigation on storage level data integrity strategies in cloud computing: Classification, security obstructions,challenges and vulnerability," Journal of Cloud Computing, vol. 13, no. 1, art. 45, 2024, doi: 10.1186/s13677-024-00605-z.

[3] J. Du, G. Dong, J. Ning, Z. Xu, et al., "Identity-based controlled delegated outsourcing data integrity auditing scheme," Scientific Reports, vol. 14, art. 7582, 2024, doi: 10.1038/s41598-024-58325-y.

[4] G. Han and H. Li, "Sec-Auditor: A blockchain-based data auditing solution for ensuring integrity and semantic correctness," Computers, Materials & Continua, vol. 80, no. 2, pp. 2121–2137, 2024, doi: 10.32604/cmc.2024.053077.

[5] H. Lei, X. A. Wang, W. Liu, L. Wu, C. Zhang, W. Jiang, and X. Zou, "An improved blockchain-based cloud auditing scheme using dynamic aggregate signatures," Computers, Materials & Continua, vol. 86, no. 2, 2026, doi: 10.32604/cmc.2025.070030.

[6] R. Li, T. Zhou, Y. Han, X. Che, W. Liu, and X. Yang, "Integrated protocol for verifiable confidential cloud computing and data integrity auditing," Journal of King Saud University Computer and Information Sciences, vol. 37, art. 299, 2025, doi: 10.1007/s44443-025-00288-9.

[7] National Institute of Standards and Technology, Digital Signature Standard (DSS), FIPS PUB 186-5, U.S. Department of Commerce, Feb. 2023, doi: 10.6028/NIST.FIPS.186-5.

[8] National Institute of Standards and Technology, Getting Started with the NIST Cybersecurity Framework: A Quick Start Guide, NIST SP 1271, 2021.

[9] Wazuh, "Wazuh documentation: File integrity monitoring and security monitoring," Wazuh Documentation.

[10] Nextcloud, "Nextcloud Server Administration Manual: Logging configuration," Nextcloud Documentation.

[11] K. A. Scarfone and M. P. Souppaya, "Guide to Computer Security Log Management," NIST Special Publication 800-92, National Institute of Standards and Technology, Gaithersburg, MD, 2006, doi: 10.6028/NIST.SP.800-92.